

Segment Selectors & Segment Registers

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- <http://creativecommons.org/licenses/by-sa/4.0/>

You are free:



to **Share** — to copy, distribute and transmit the work



to **Remix** — to adapt the work

Under the following conditions:



Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).



Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Segmentation

- “Segmentation provides a mechanism for dividing the processor’s addressable memory space (called the **linear address space**) into smaller protected address spaces called **segments**.”
(emphasis theirs)

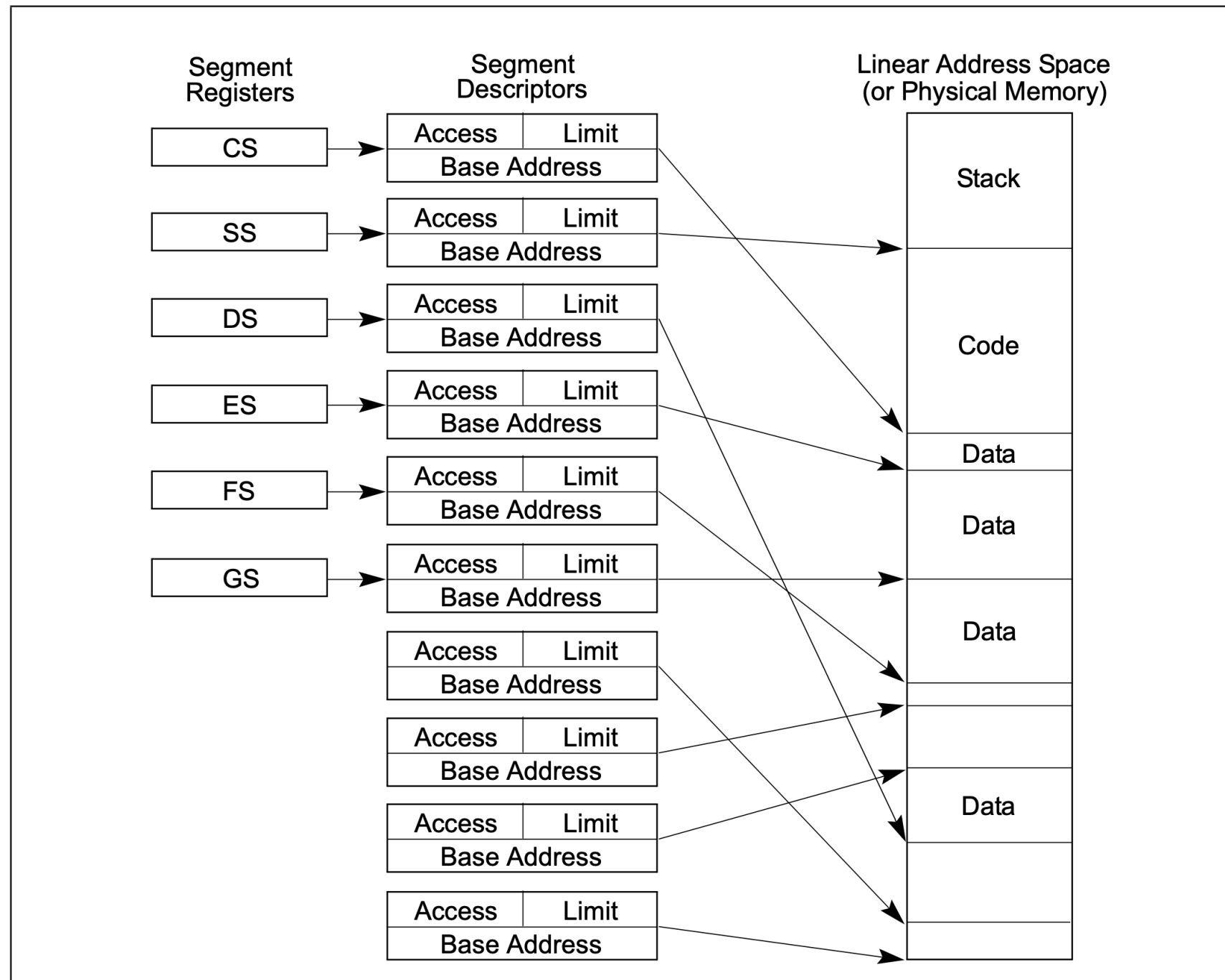


Figure 3-4. Multi-Segment Model

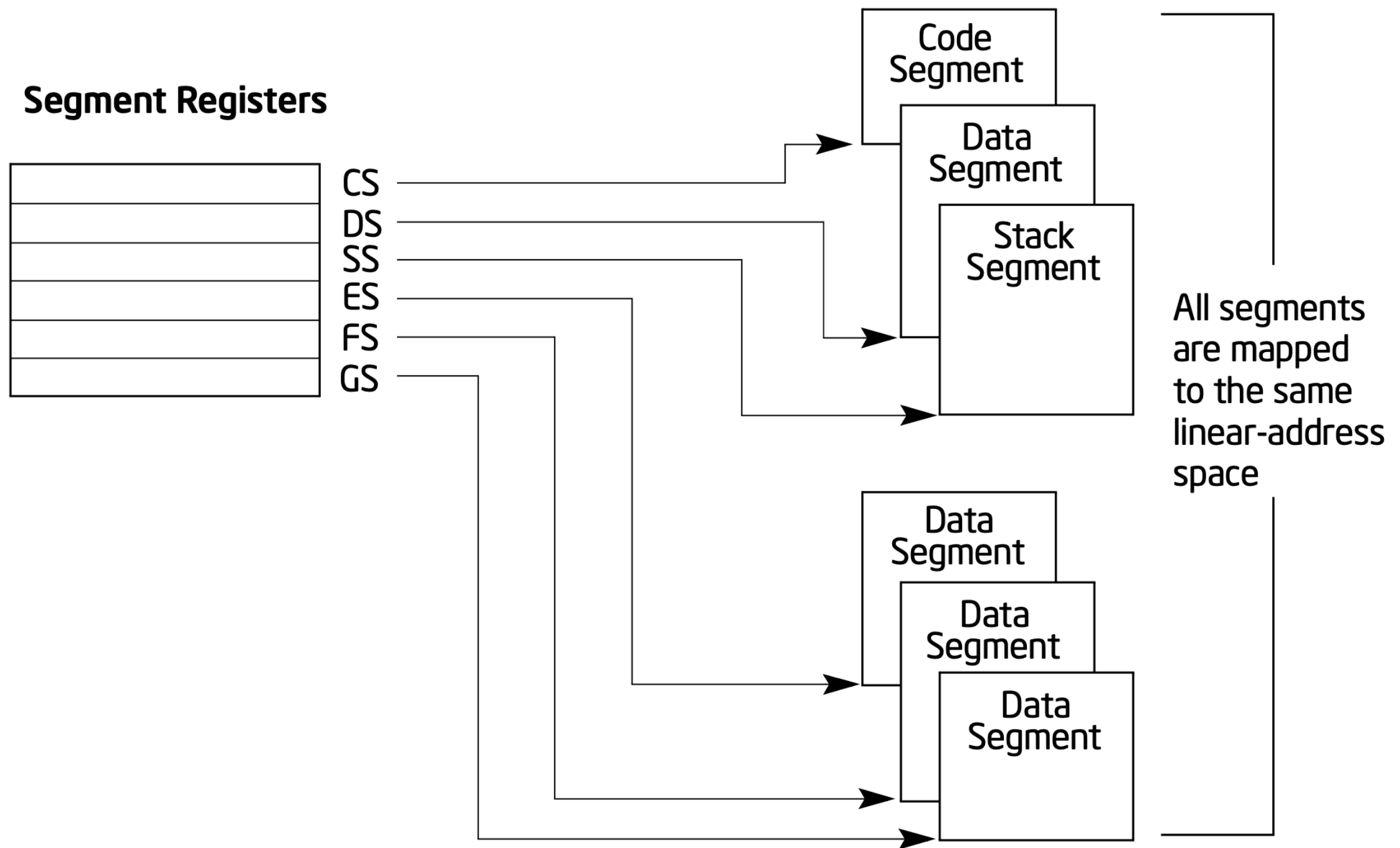
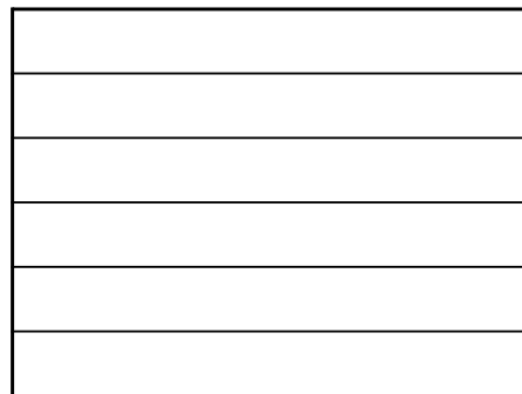


Figure 3-7. Use of Segment Registers in Segmented Memory Model

Segment Registers



CS
DS
SS
ES
FS
GS

The segment selector in each segment register points to an overlapping segment in the linear address space.

Linear Address Space for Program

Overlapping
Segments
of up to
4 GBytes
Beginning at
Address 0

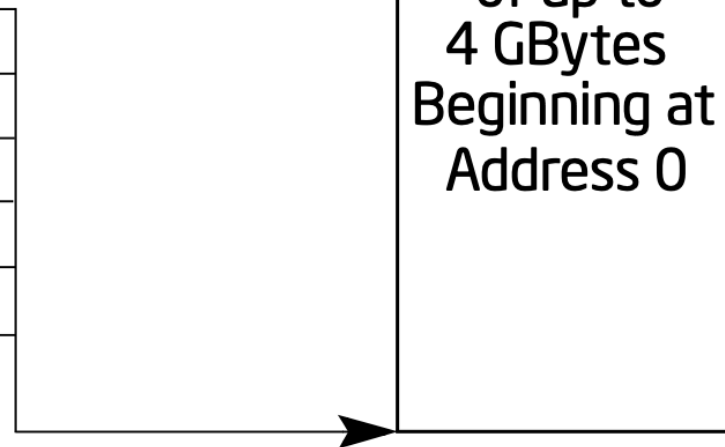


Figure 3-6. Use of Segment Registers for Flat Memory Model

Segment Addressing

- "There is no mode bit to disable segmentation."
- "To locate a byte in a particular segment, a **logical address** (also called a far pointer) must be provided. A logical address consists of a segment selector and an offset."
- "The physical address space is defined as the range of addresses that the processor can generate on its address bus"
- Linear address space is a flat 64 bit space

32-bit Near Pointer vs. Far Pointer

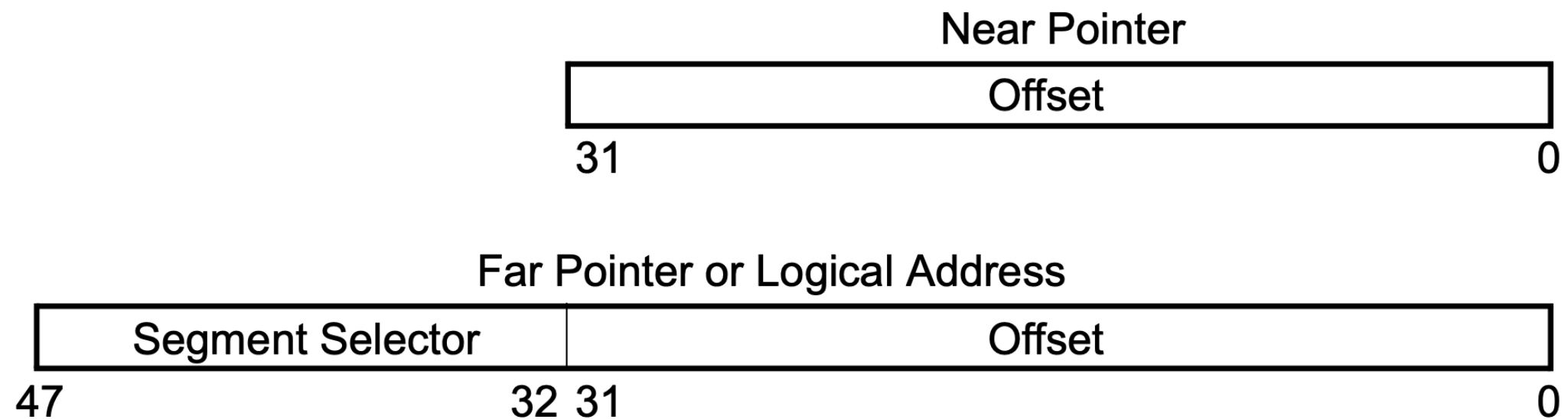


Figure 4-4. Pointer Data Types

64-bit Near Pointer vs. Far Pointer

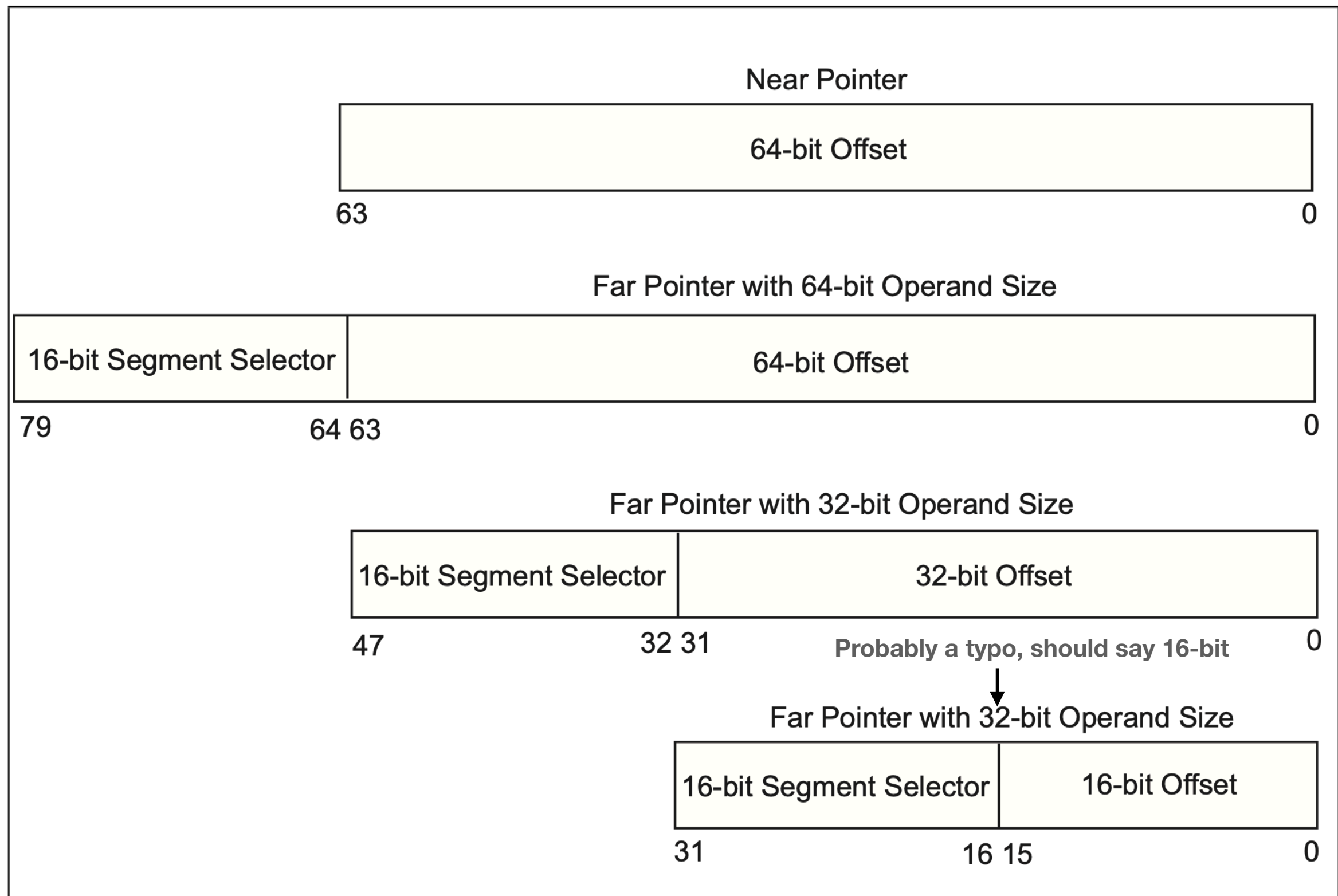


Figure 4-5. Pointers in 64-Bit Mode

64-bit Near Pointer vs. Far Pointer

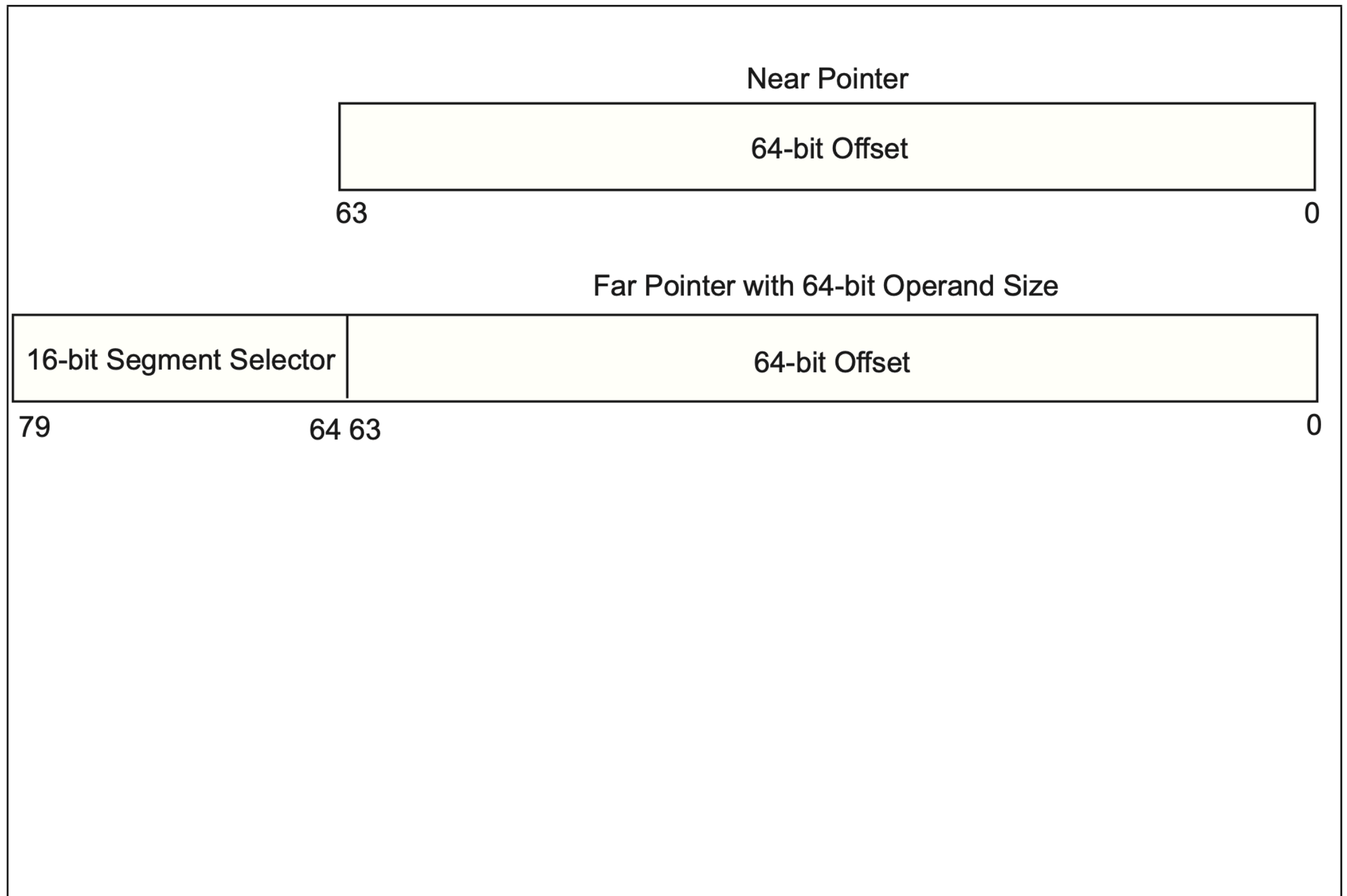
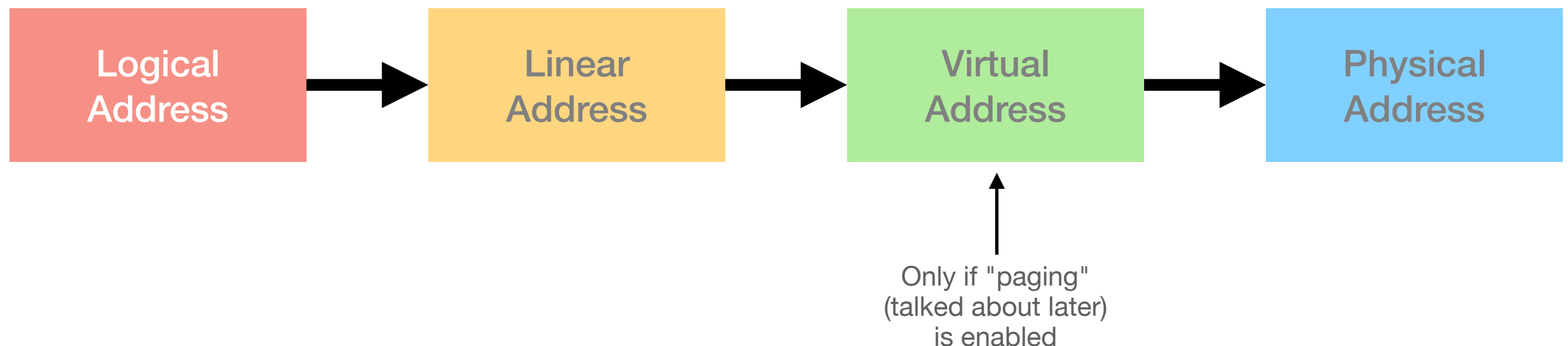


Figure 4-5. Pointers in 64-Bit Mode

Address Spaces Restated 1

- Logical Address: Far pointer that specifies a segment selector + offset in that segment
- Linear Address: Flat 32/64-bit notional address-space, post-segmentation
- Virtual Address: Address that takes paging into account
- Physical Address: Offset into physical RAM



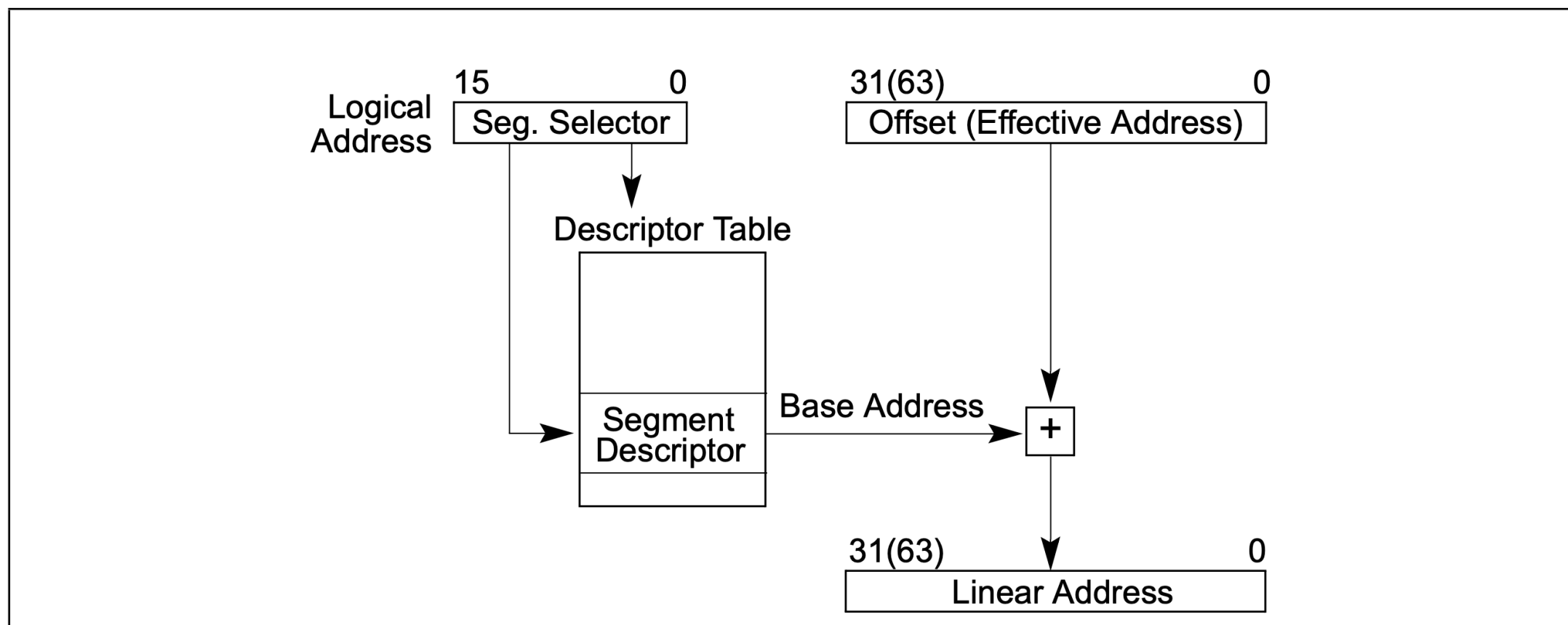
Address Spaces Restated 2

- We haven't learned about paging yet, so let's pretend it's disabled (*which it is* when the processor first starts up)
- In which case Linear addresses == Physical addresses



Segmentation Restated 1

- Segmentation translates logical addresses to linear addresses, automatically, in hardware, by using table lookups
 - But software fills in the tables
- Logical address (far pointer) = 16 bit segment selector + 32/64 bit offset



Segmentation Restated 2

- Segmentation translates logical addresses to linear addresses, automatically, in hardware, by using table lookups
 - But software fills in the tables
- Logical address (far pointer) = 16 bit segment selector + 32/64 bit offset

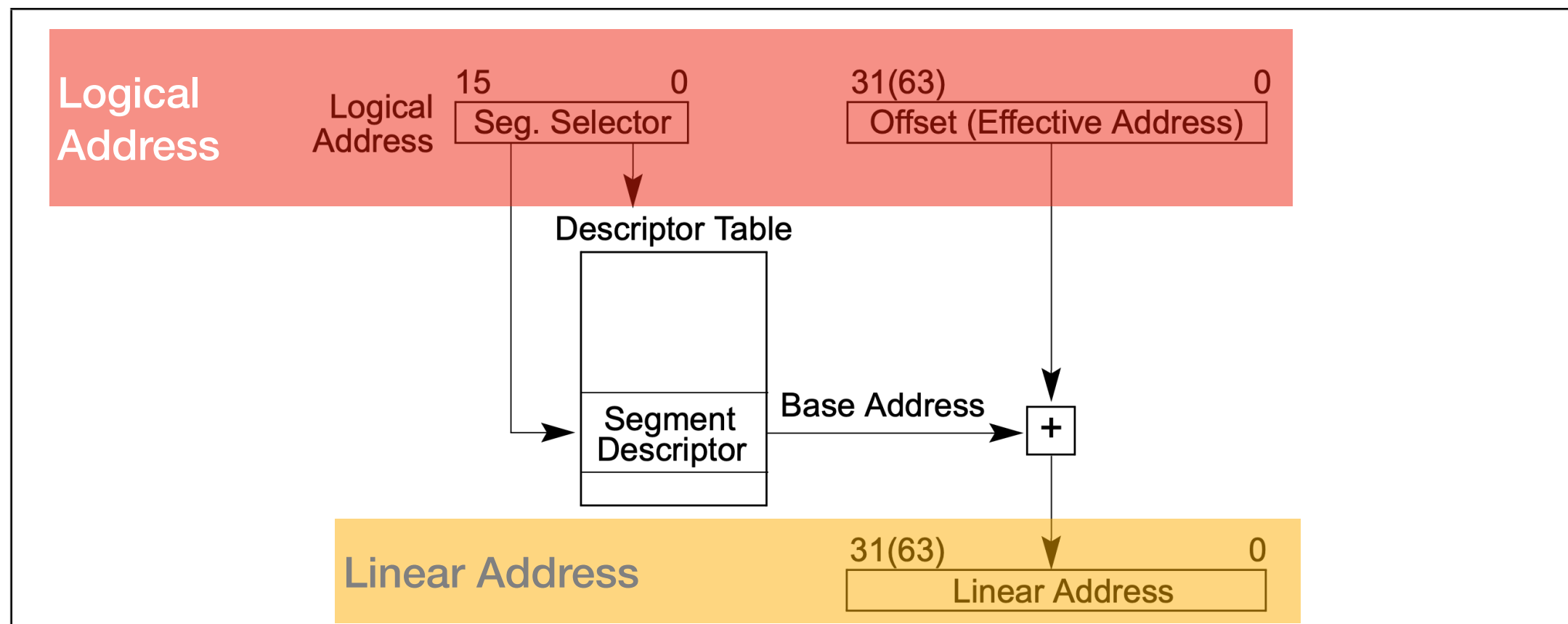


Figure 3-5. Logical Address to Linear Address Translation

The Big Picture 1

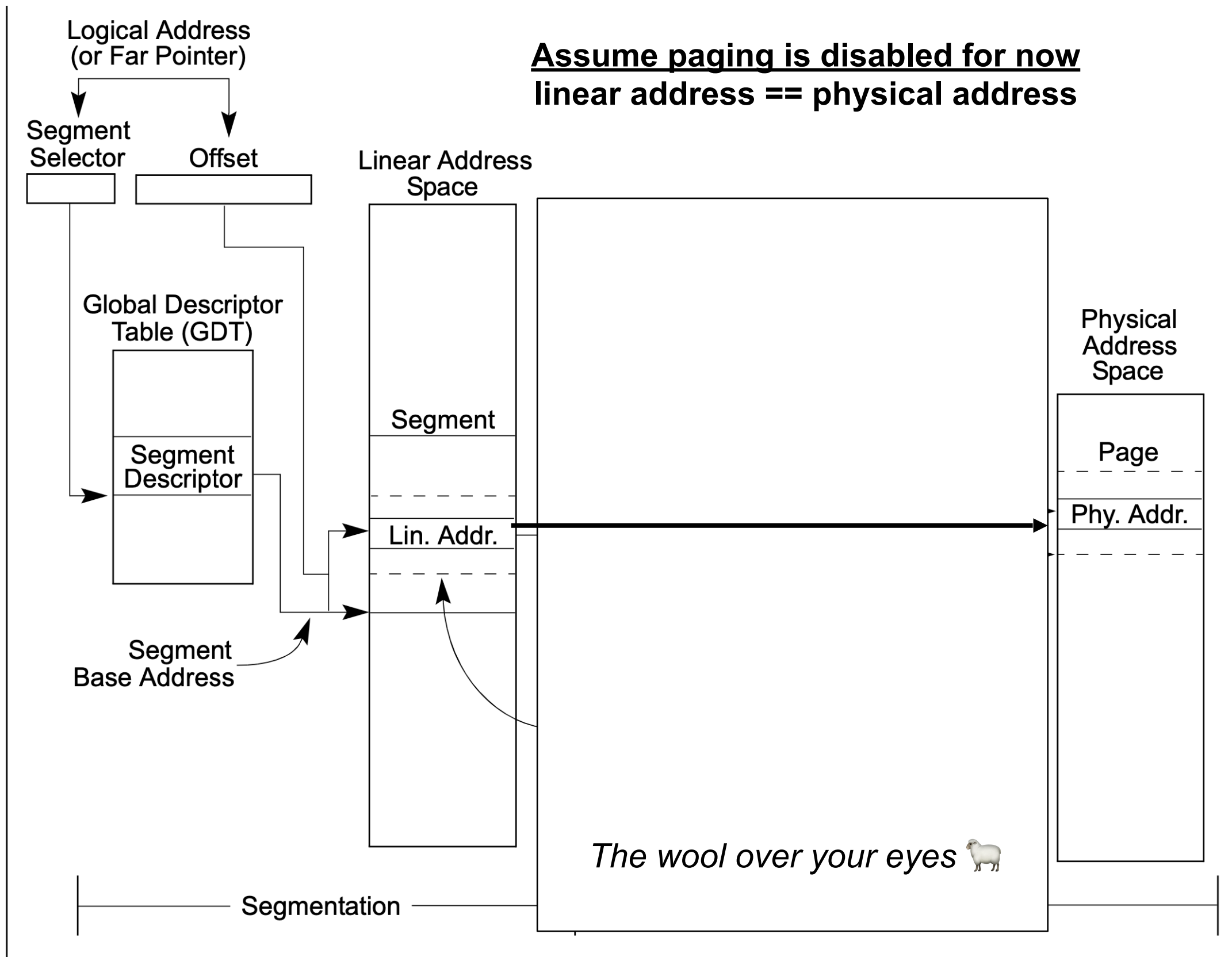


Figure 3-1. Segmentation and Paging

The Big Picture 2

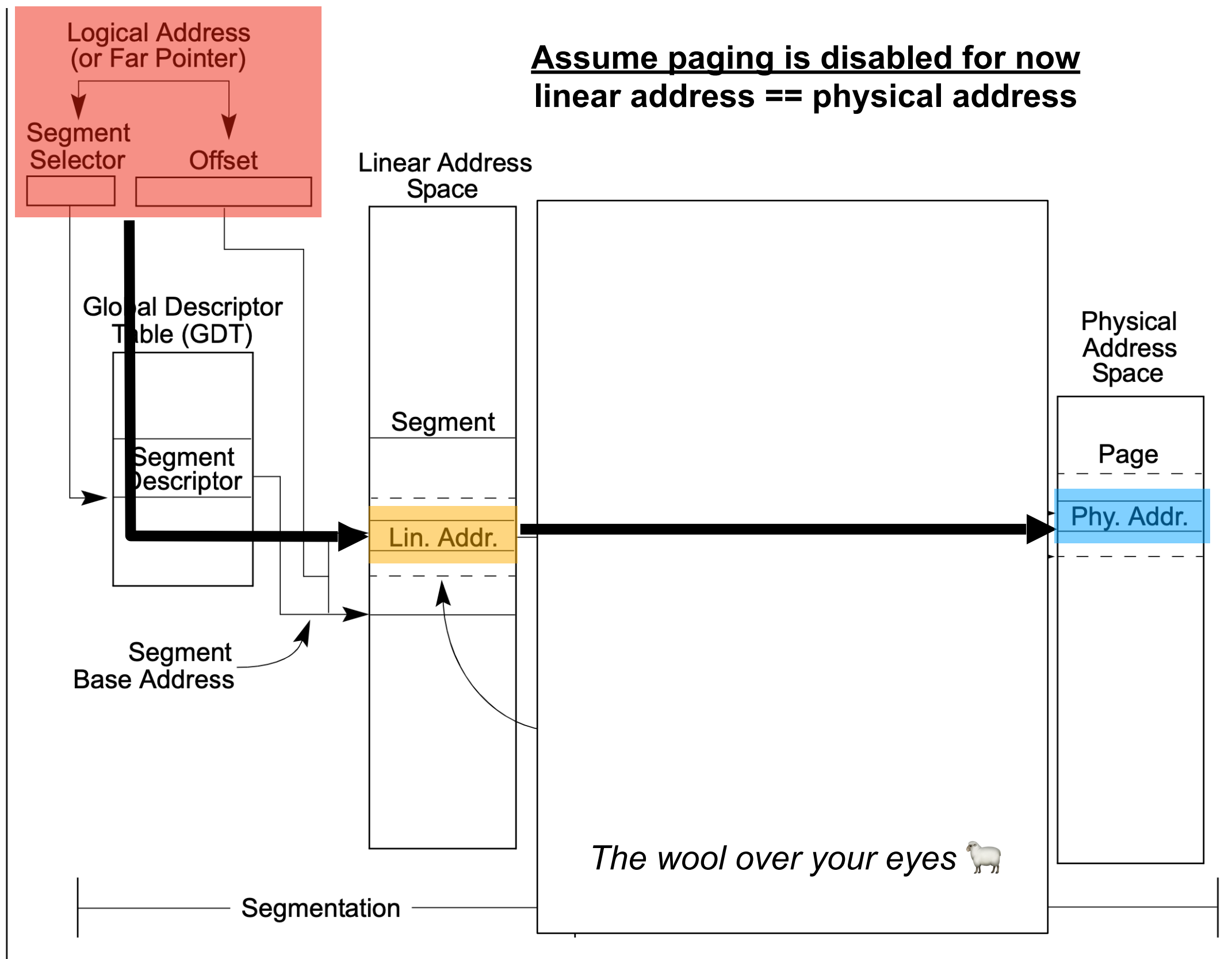


Figure 3-1. Segmentation and Paging

x86-64 Codifies What Everyone Was Already Doing...

3.2.4 Segmentation in IA-32e Mode

In IA-32e mode of Intel 64 architecture, the effects of segmentation depend on whether the processor is running in compatibility mode or 64-bit mode. In compatibility mode, segmentation functions just as it does using legacy 16-bit or 32-bit protected mode semantics.

In 64-bit mode, segmentation is generally (but not completely) disabled, creating a flat 64-bit linear-address space. The processor treats the segment base of CS, DS, ES, SS as zero, creating a linear address that is equal to the effective address. The FS and GS segments are exceptions. These segment registers (which hold the segment base) can be used as additional base registers in linear address calculations. They facilitate addressing local data and certain operating system data structures.

Note that the processor does not perform segment limit checks at runtime in 64-bit mode.

Not This...

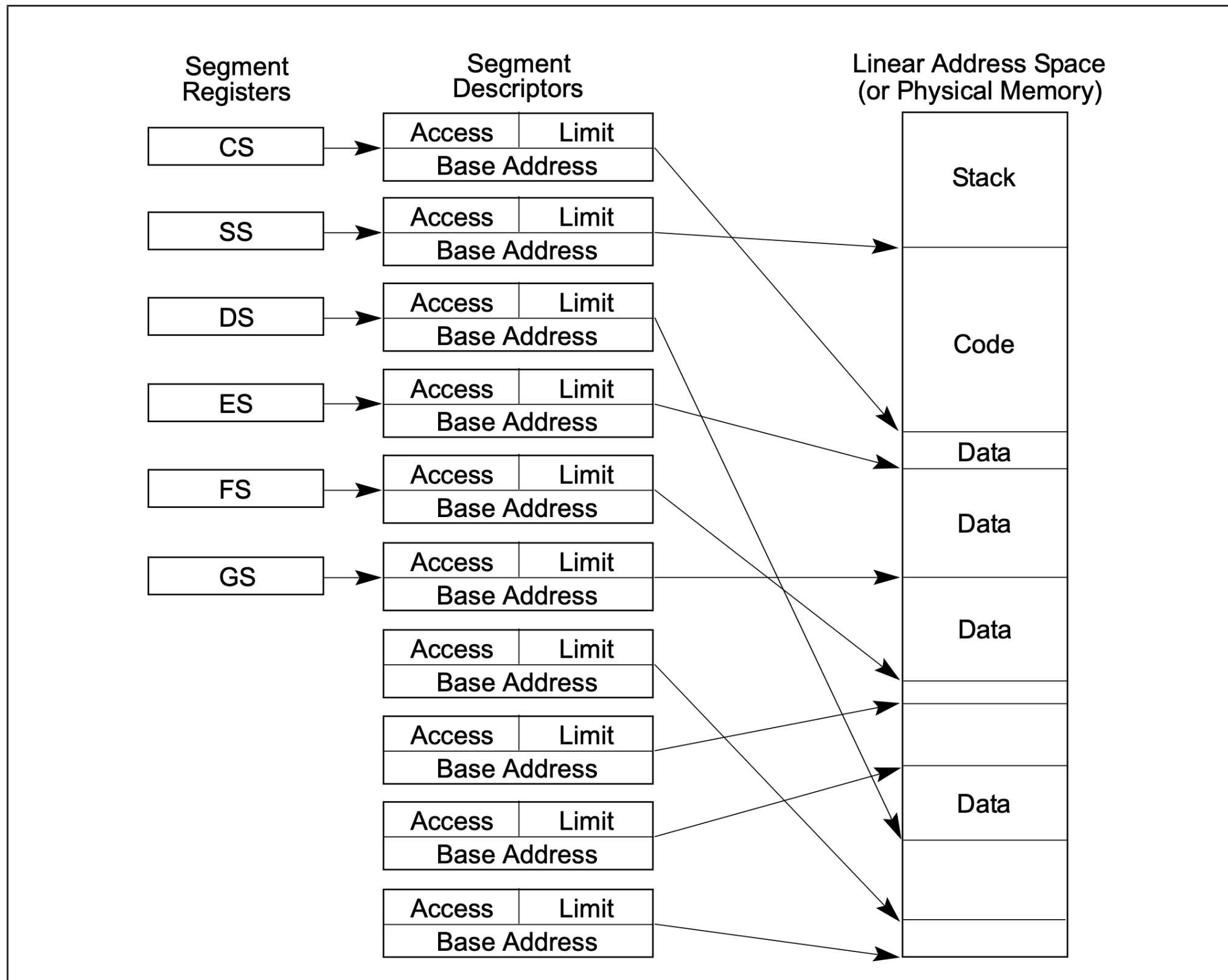


Figure 3-4. Multi-Segment Model

Not This....

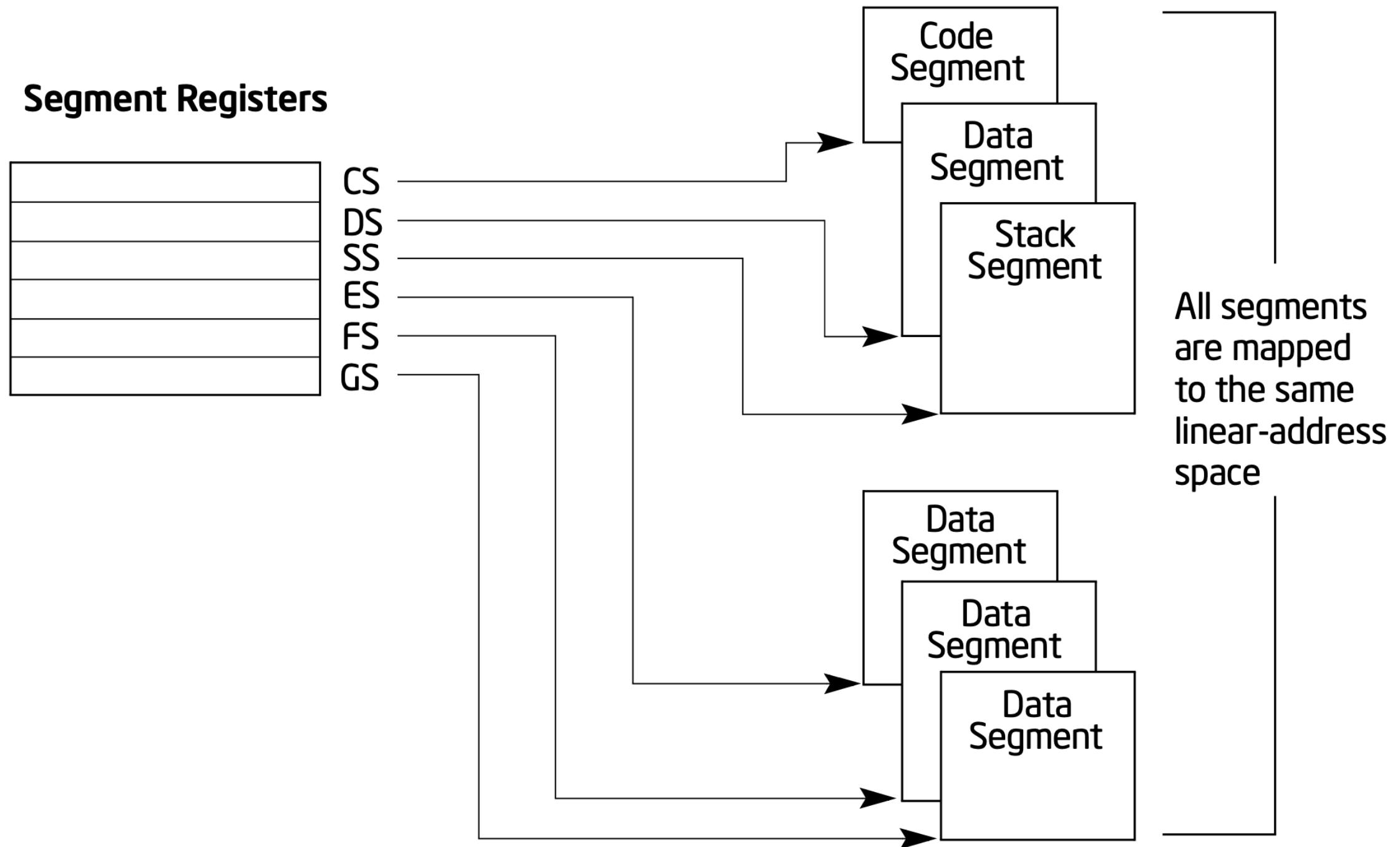


Figure 3-7. Use of Segment Registers in Segmented Memory Model

Not This...

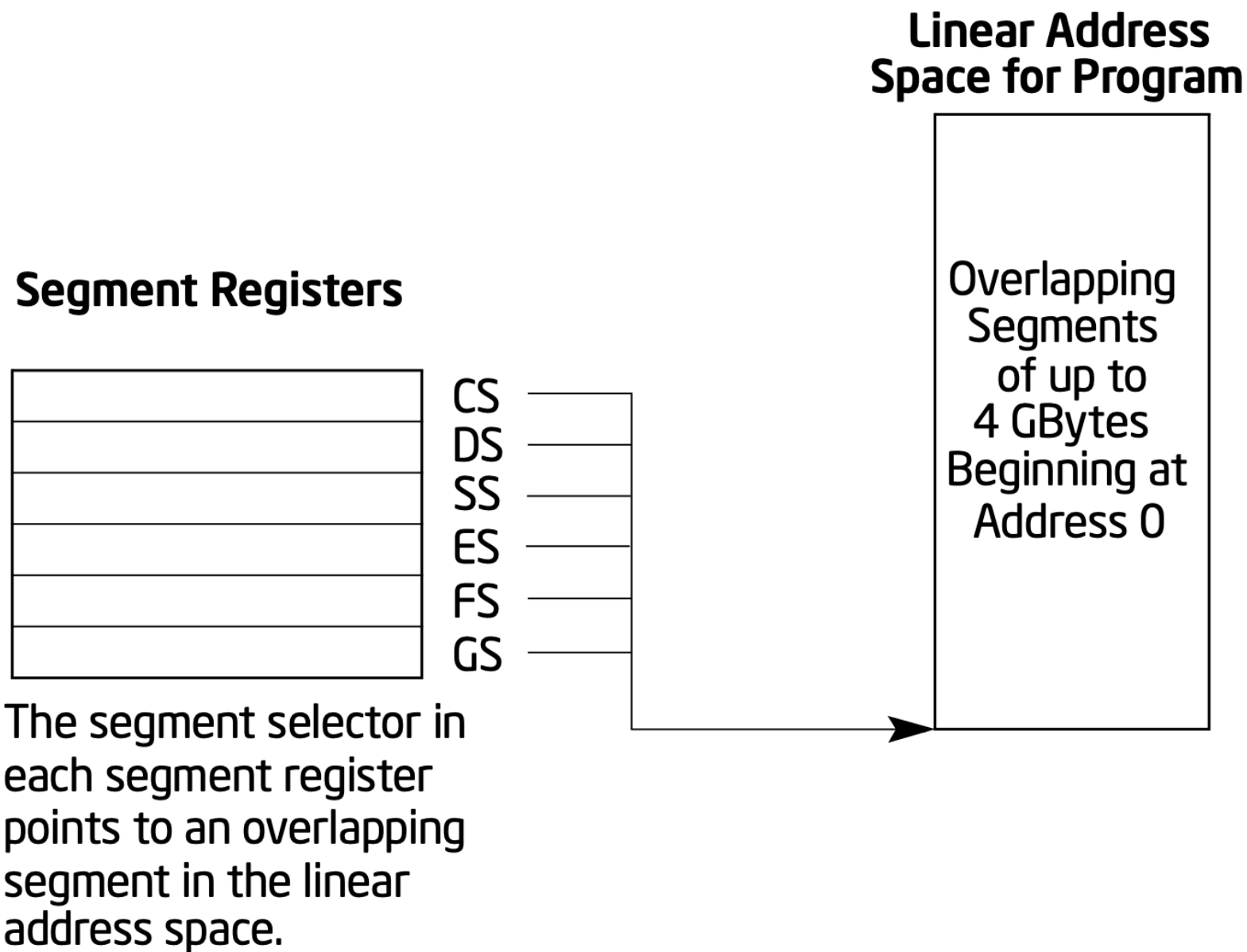
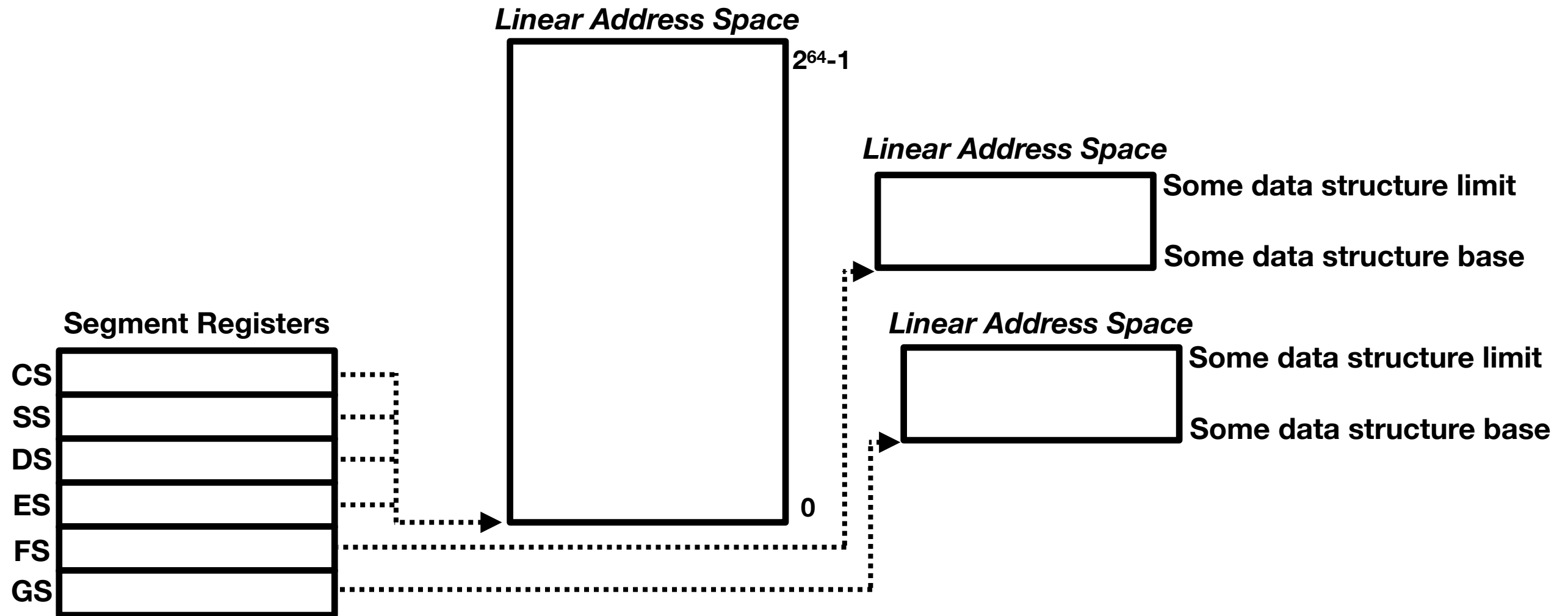


Figure 3-6. Use of Segment Registers for Flat Memory Model

But This...



64-bit Near Pointer vs. Far Pointer

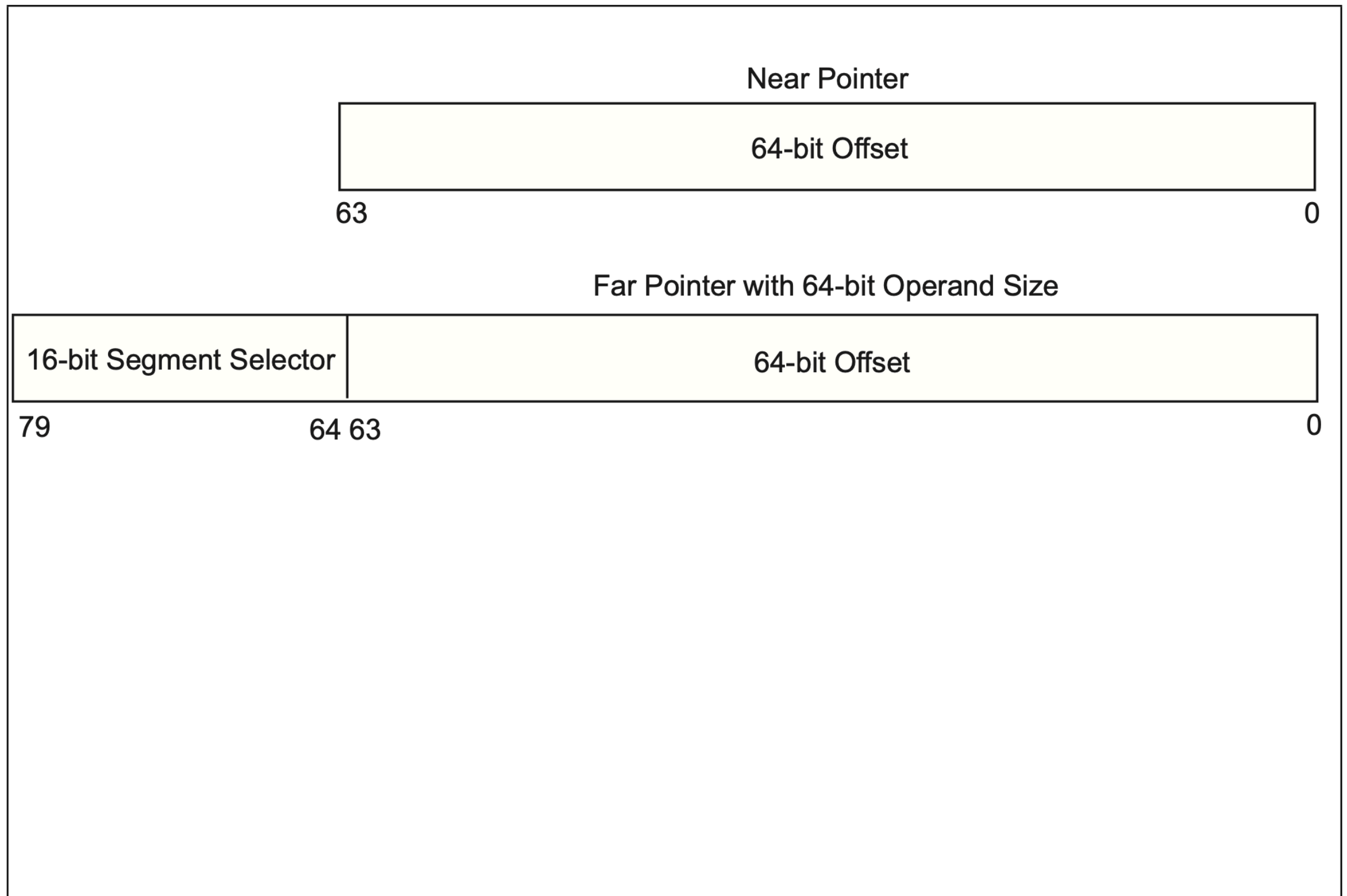
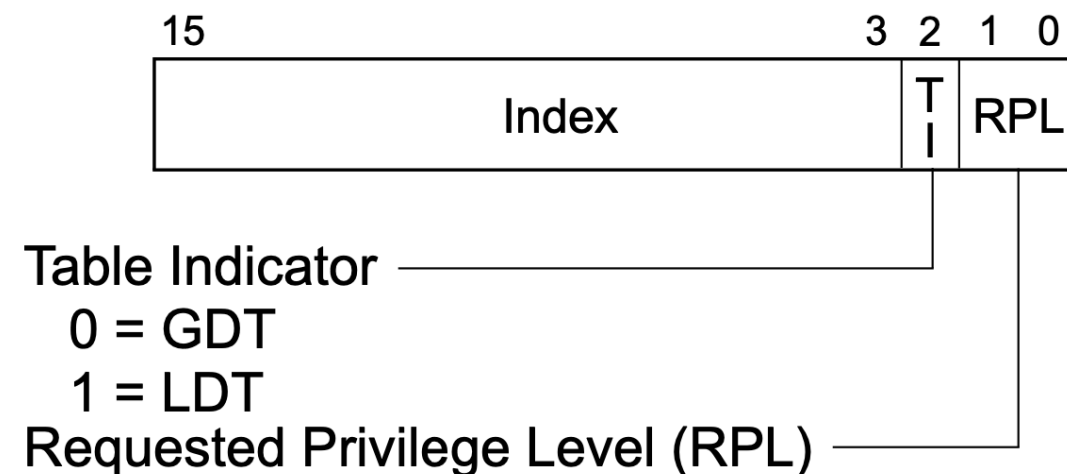


Figure 4-5. Pointers in 64-Bit Mode

64-bit Near Pointer vs. Far Pointer



Segment Selectors

- A **segment selector** is a 16 bit data structure. It is used to select a different data structure from one of two tables, that are covered more in the next unit
 - GDT - Global Descriptor Table
 - LDT - Local Descriptor Table
- Note that the table index is actually 13 bits not 16, so the tables can each hold a maximum of $2^{13} = 8192$ data structures

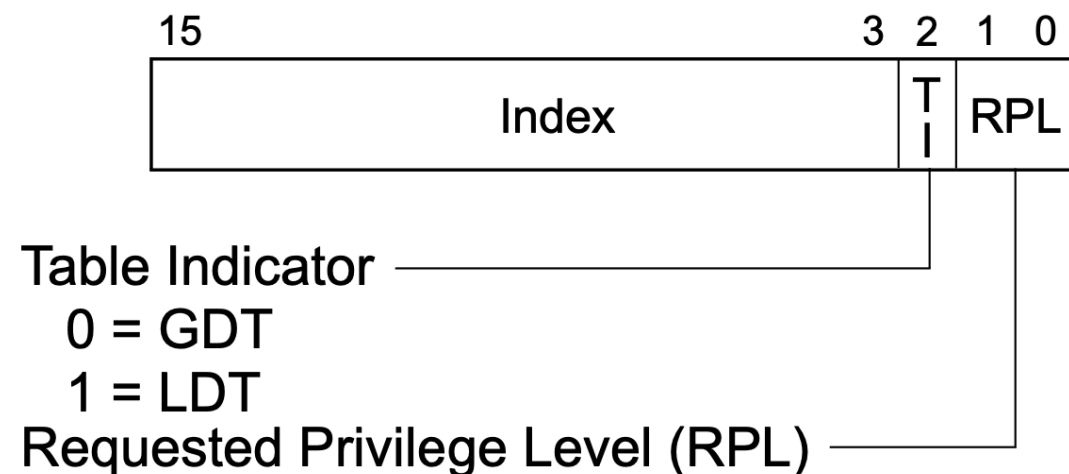
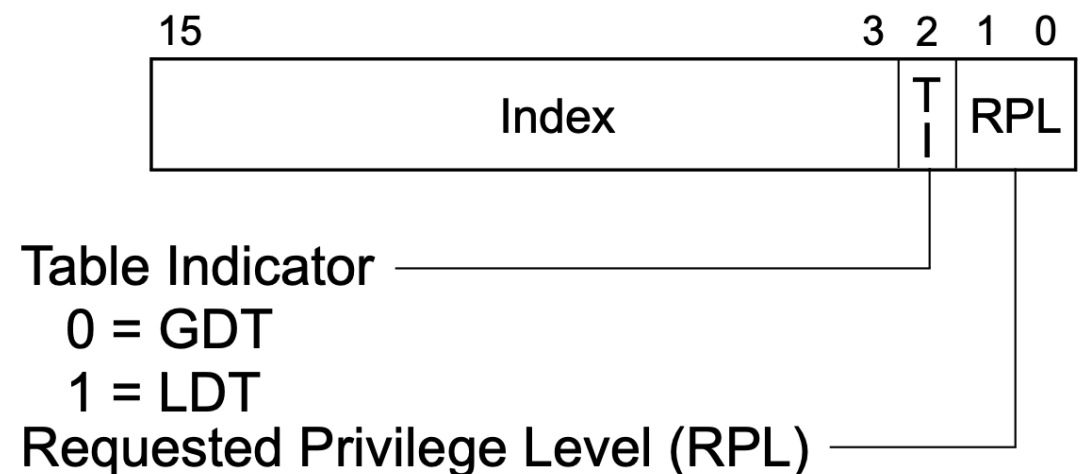


Figure 3-6. Segment Selector

Segment Selectors

- A **segment selector** is a 16 bit data structure. It is used to select a different data structure from one of two tables, that are covered more in the next unit
 - GDT - Global Descriptor Table
 - LDT - Local Descriptor Table
- Note that the table index is actually 13 bits not 16, so the tables can each hold a maximum of $2^{13} = 8192$ data structures



2 bit “privilege level”?
Hmmm...getting warm
push that onto a
mental stack

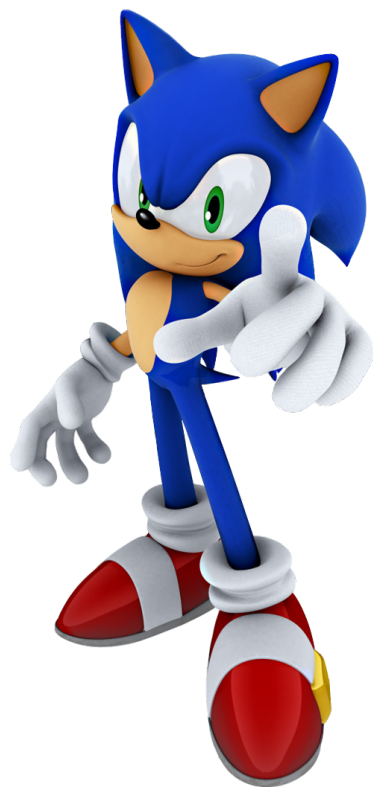


Figure 3-6. Segment Selector

The Six Segment Registers

- Six 16-bit registers that hold a *Segment Selector*
- CS - Code Segment
- SS - Stack Segment
- DS - Data Segment
- ES/FS/GS - Extra (usually data) segment registers

Arch1001

Arch2001

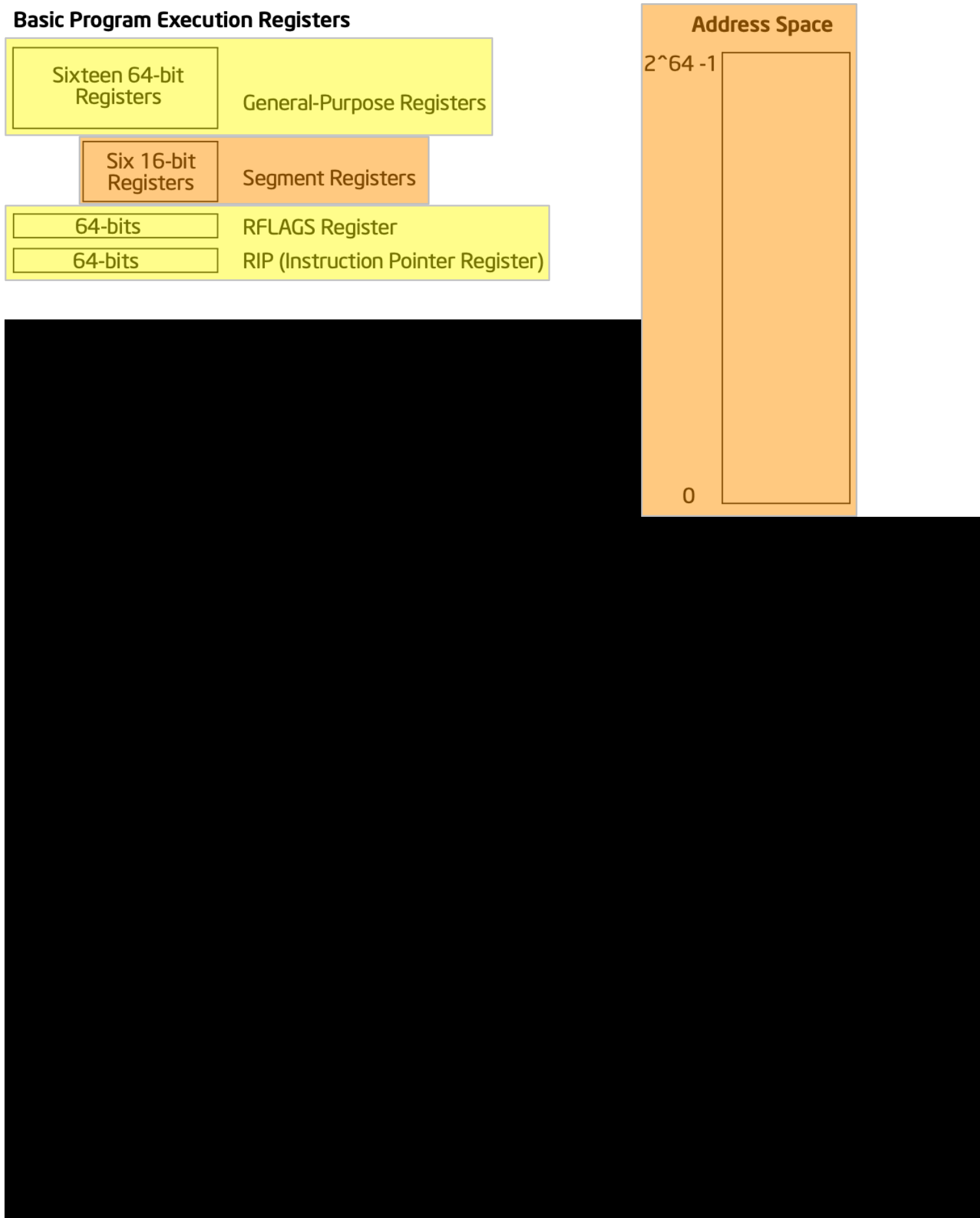
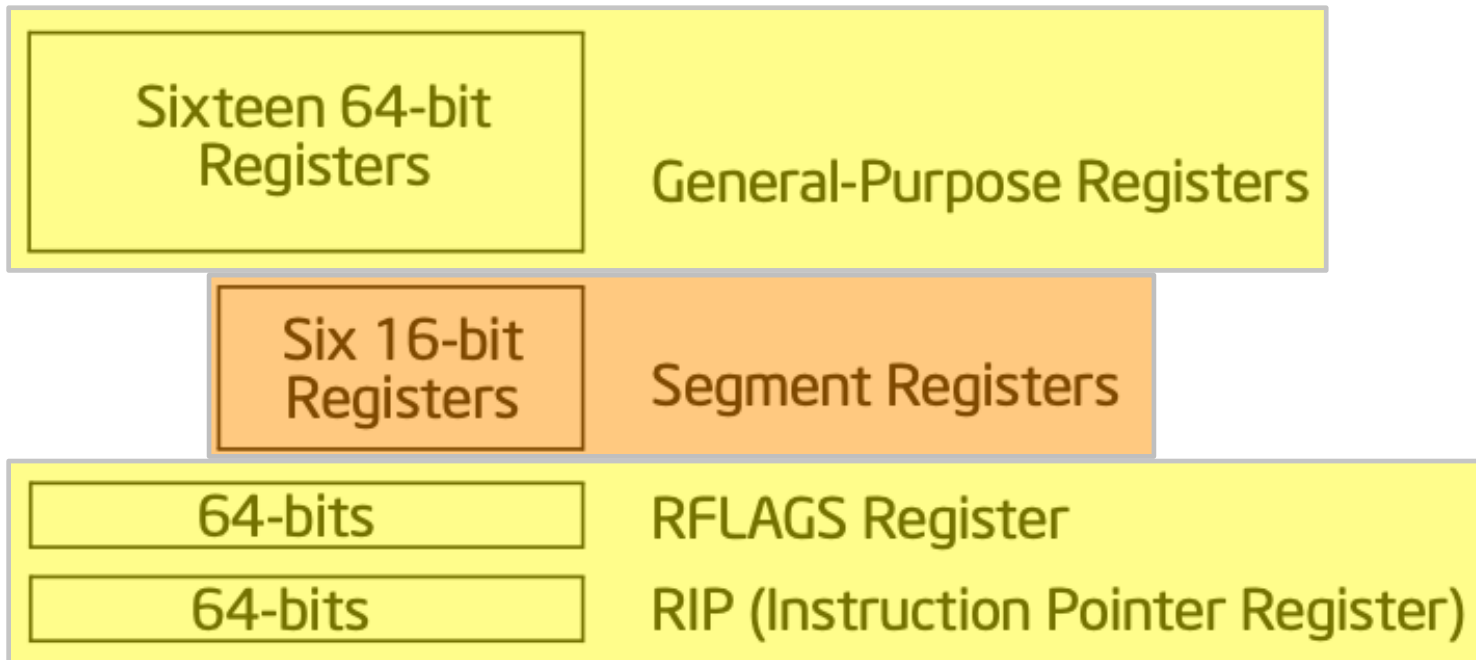


Figure 3-2. 64-Bit Mode Execution Environment

Arch1001

Arch2001

Basic Program Execution Registers



Address Space

$2^{64} - 1$

0

Reading/Writing Segment Registers With MOV

8C / <i>r</i>	MOV <i>r/m16, Sreg**</i>	MR	Valid	Valid	Move segment register to <i>r/m16</i> .
8C / <i>r</i>	MOV <i>r16/r32/m16, Sreg**</i>	MR	Valid	Valid	Move zero extended 16-bit segment register to <i>r16/r32/m16</i> .
REX.W + 8C / <i>r</i>	MOV <i>r64/m16, Sreg**</i>	MR	Valid	Valid	Move zero extended 16-bit segment register to <i>r64/m16</i> .
8E / <i>r</i>	MOV <i>Sreg, r/m16**</i>	RM	Valid	Valid	Move <i>r/m16</i> to segment register.
REX.W + 8E / <i>r</i>	MOV <i>Sreg, r/m64**</i>	RM	Valid	Valid	Move <i>lower 16 bits of r/m64</i> to segment register.

- "MOV *SR, r/m16*" and "MOV *r/m16, SR*"

Reading/Writing Segment Registers With MOV

8C / <i>r</i>	MOV <i>r/m16</i> , <i>Sreg</i> **	MR	Valid	Valid	Move segment register to <i>r/m16</i> .
8C / <i>r</i>	MOV <i>r16/r32/m16</i> , <i>Sreg</i> **	MR	Valid	Valid	Move zero extended 16-bit segment register to <i>r16/r32/m16</i> .
REX.W + 8C / <i>r</i>	MOV <i>r64/m16</i> , <i>Sreg</i> **	MR	Valid	Valid	Move zero extended 16-bit segment register to <i>r64/m16</i> .
8E / <i>r</i>	MOV <i>Sreg</i> , <i>r/m16</i> **	RM	Valid	Valid	Move <i>r/m16</i> to segment register.
REX.W + 8E / <i>r</i>	MOV <i>Sreg</i> , <i>r/m64</i> **	RM	Valid	Valid	Move <i>lower 16 bits of r/m64</i> to segment register.

★₆ • "MOV *SR*, *r/m16*" and "MOV *r/m16*, *SR*"

Reading/Writing Segment Registers With PUSH/POP

0E	PUSH CS	Z0	Invalid	Valid	Push CS.
16	PUSH SS	Z0	Invalid	Valid	Push SS.
1E	PUSH DS	Z0	Invalid	Valid	Push DS.
06	PUSH ES	Z0	Invalid	Valid	Push ES.
0F A0	PUSH FS	Z0	Valid	Valid	Push FS.
0F A8	PUSH GS	Z0	Valid	Valid	Push GS.

- PUSH *SR***

1F	POP DS	Z0	Invalid	Valid	Pop top of stack into DS; increment stack pointer.
07	POP ES	Z0	Invalid	Valid	Pop top of stack into ES; increment stack pointer.
17	POP SS	Z0	Invalid	Valid	Pop top of stack into SS; increment stack pointer.
0F A1	POP FS	Z0	Valid	Valid	Pop top of stack into FS; increment stack pointer by 16 bits.
0F A1	POP FS	Z0	N.E.	Valid	Pop top of stack into FS; increment stack pointer by 32 bits.
0F A1	POP FS	Z0	Valid	N.E.	Pop top of stack into FS; increment stack pointer by 64 bits.
0F A9	POP GS	Z0	Valid	Valid	Pop top of stack into GS; increment stack pointer by 16 bits.
0F A9	POP GS	Z0	N.E.	Valid	Pop top of stack into GS; increment stack pointer by 32 bits.
0F A9	POP GS	Z0	Valid	N.E.	Pop top of stack into GS; increment stack pointer by 64 bits.

Note: no "pop CS"

- POP *SR***

Reading/Writing Segment Registers With PUSH/POP

0E	PUSH CS	Z0	Invalid	Valid	Push CS.
16	PUSH SS	Z0	Invalid	Valid	Push SS.
1E	PUSH DS	Z0	Invalid	Valid	Push DS.
06	PUSH ES	Z0	Invalid	Valid	Push ES.
0F A0	PUSH FS	Z0	Valid	Valid	Push FS.
0F A8	PUSH GS	Z0	Valid	Valid	Push GS.



• PUSH SR

1F	POP DS	Z0	Invalid	Valid	Pop top of stack into DS; increment stack pointer.
07	POP ES	Z0	Invalid	Valid	Pop top of stack into ES; increment stack pointer.
17	POP SS	Z0	Invalid	Valid	Pop top of stack into SS; increment stack pointer.
0F A1	POP FS	Z0	Valid	Valid	Pop top of stack into FS; increment stack pointer by 16 bits.
0F A1	POP FS	Z0	N.E.	Valid	Pop top of stack into FS; increment stack pointer by 32 bits.
0F A1	POP FS	Z0	Valid	N.E.	Pop top of stack into FS; increment stack pointer by 64 bits.
0F A9	POP GS	Z0	Valid	Valid	Pop top of stack into GS; increment stack pointer by 16 bits.
0F A9	POP GS	Z0	N.E.	Valid	Pop top of stack into GS; increment stack pointer by 32 bits.
0F A9	POP GS	Z0	Valid	N.E.	Pop top of stack into GS; increment stack pointer by 64 bits.

Note: no "pop CS"



• POP SR

Segment Registers Have a Hidden Side...

- Normally when we're talking about segment registers, we're only talking about the 16-bit visible part
- There's also a "hidden" part that you can't directly access
 - It's like a cache so that translation table info doesn't have to be looked up from RAM each time. (Vol 3, Sect 3.4.4)

Visible Part		Hidden Part	
Segment Selector		Base Address, Limit, Access Information	
			CS
			SS
			DS
			ES
			FS
			GS

Figure 3-7. Segment Registers

What's Really Happening With x86-64 Segmentation...

Visible Part		Hidden Part	
Segment Selector	Base Address, Limit, Access Information		CS
			SS
			DS
			ES
			FS
			GS

Figure 3-7. Segment Registers

What's Really Happening With x86-64 Segmentation...

Visible Part

Hidden Part

Segment Selector	Base Address, Limit, Access Information	CS
		SS
		DS
		ES
		FS
		GS

Figure 3-7. Segment Registers

What's Really Happening With x86-64 Segmentation...

Visible Part

Hidden Part

Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	CS
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	SS
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	DS
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	ES
Segment Selector	<i><u>Base</u> From Table</i>	<i><u>Limit</u> From Table</i>	<i><u>Access</u> From Table</i>	FS
Segment Selector	<i><u>Base</u> From Table</i>	<i><u>Limit</u> From Table</i>	<i><u>Access</u> From Table</i>	GS

What's Really Happening With x86-64 Segmentation...

Visible Part	Hidden Part			
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	CS
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	SS
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	DS
Segment Selector	<i>Hardcoded Base == 0</i>	<i>Hardcoded Limit == 2⁶⁴-1</i>	<i><u>Access</u> From Table</i>	ES
Segment Selector	<i><u>Base</u> From Table</i>	<i><u>Limit</u> From Table</i>	<i><u>Access</u> From Table</i>	FS
Segment Selector	<i><u>Base</u> From Table</i>	<i><u>Limit</u> From Table</i>	<i><u>Access</u> From Table</i>	GS

What's Really Happening With x86-64 Segmentation...

CS



Segment Selector

A diagram showing a rectangular box with a black border. The text "Segment Selector" is centered inside the box. To the left of the box, the text "CS" is displayed.

What's Really Happening With x86-64 Segmentation...

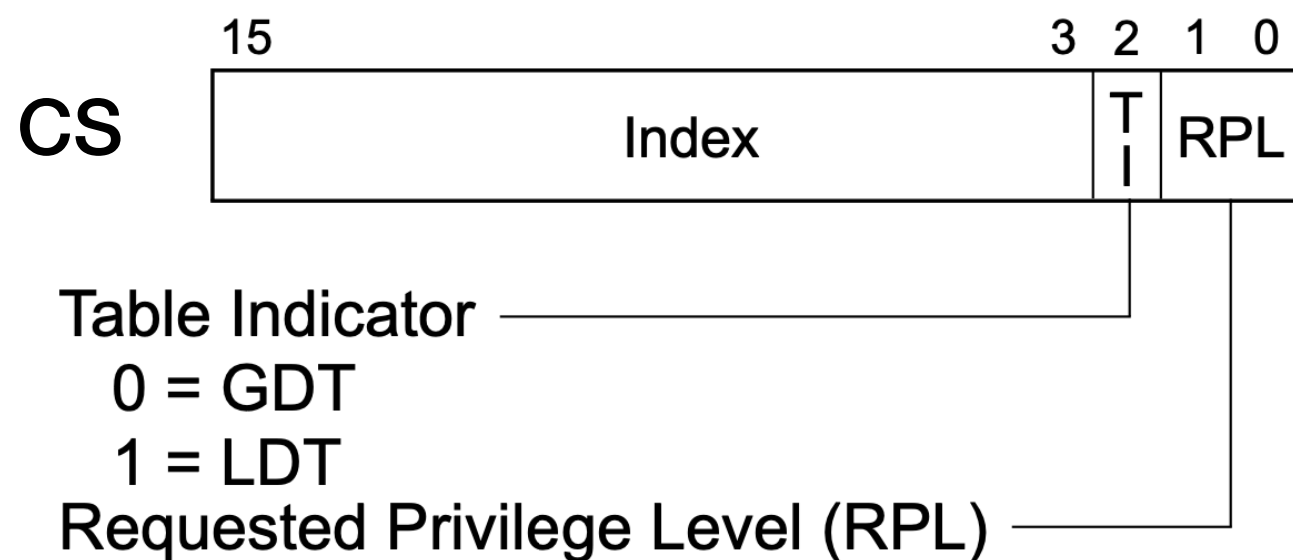


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

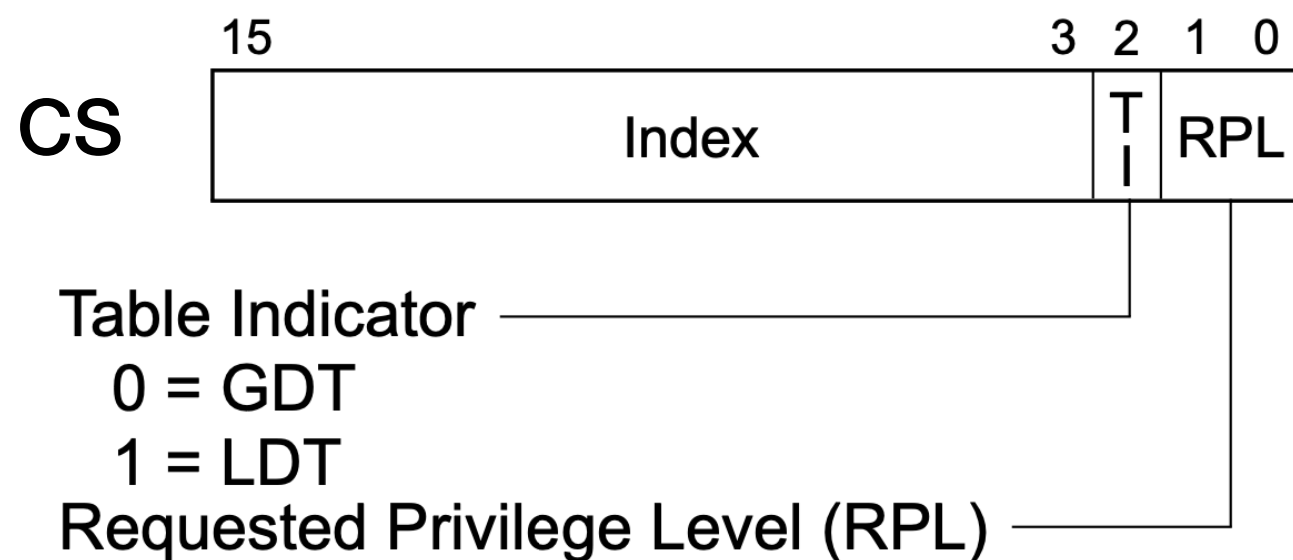


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

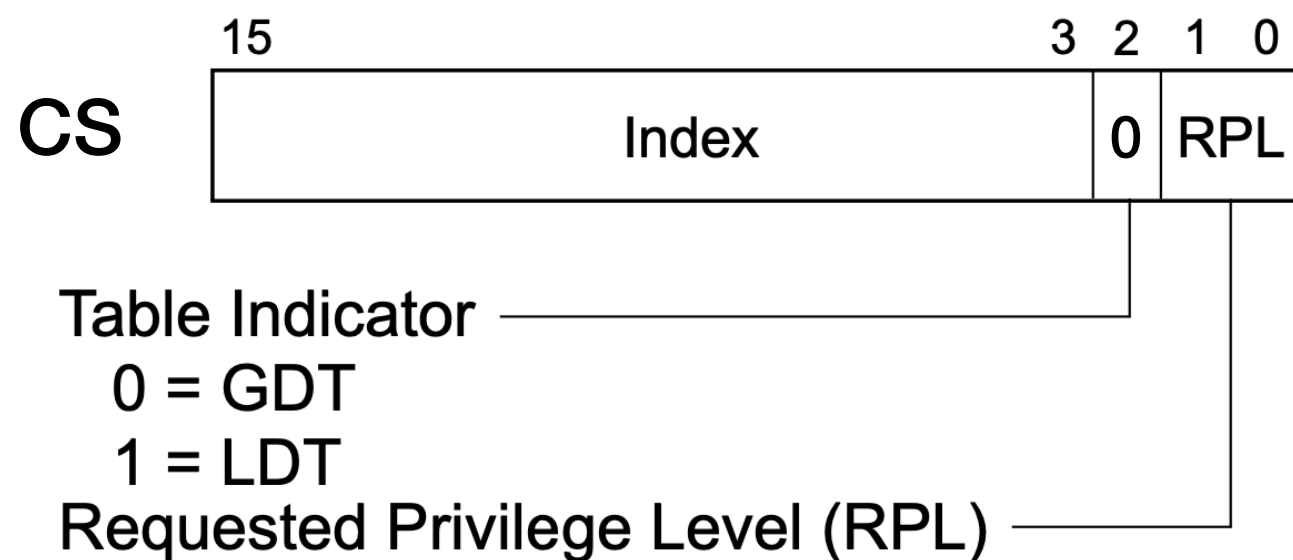


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...



Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

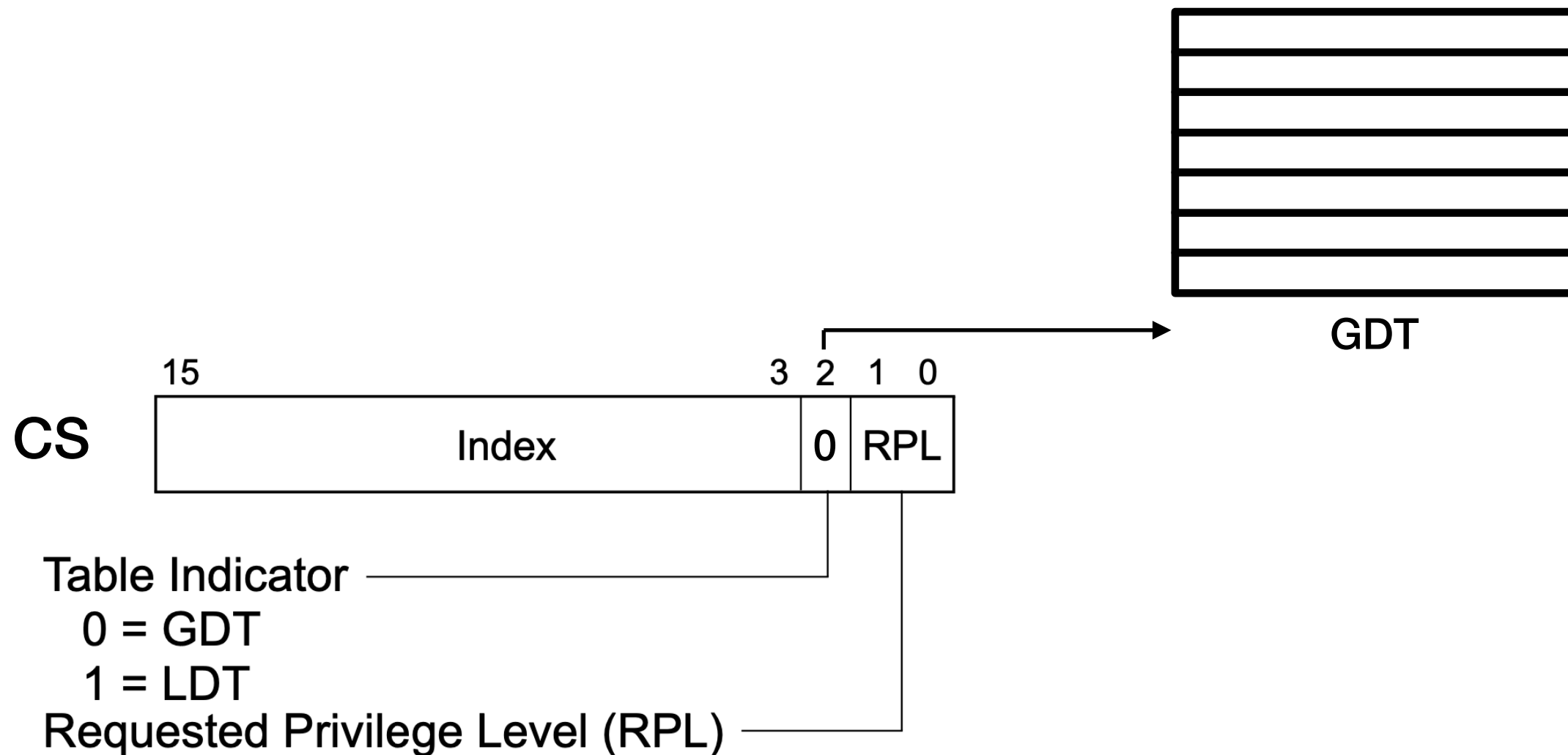


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

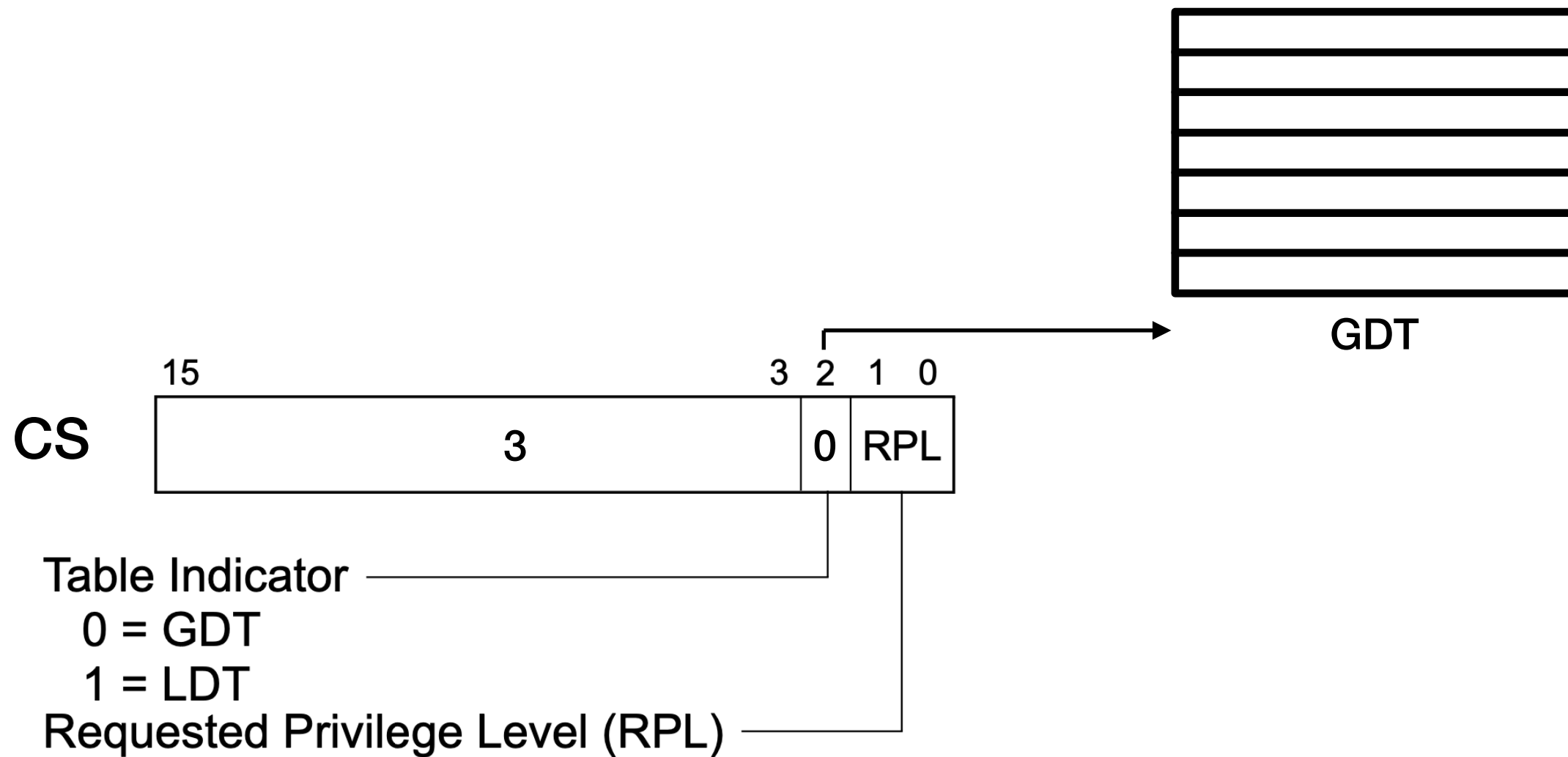


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

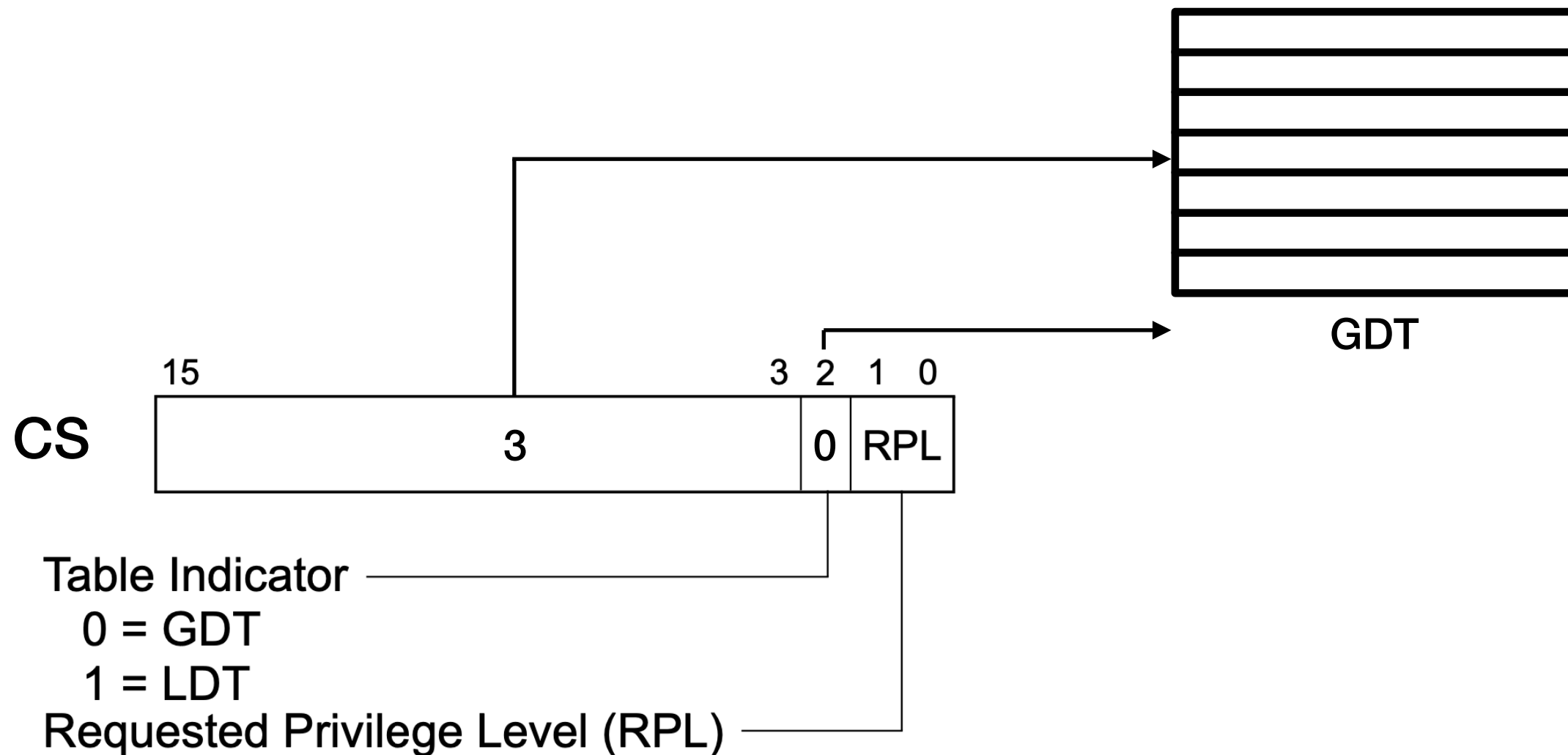


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

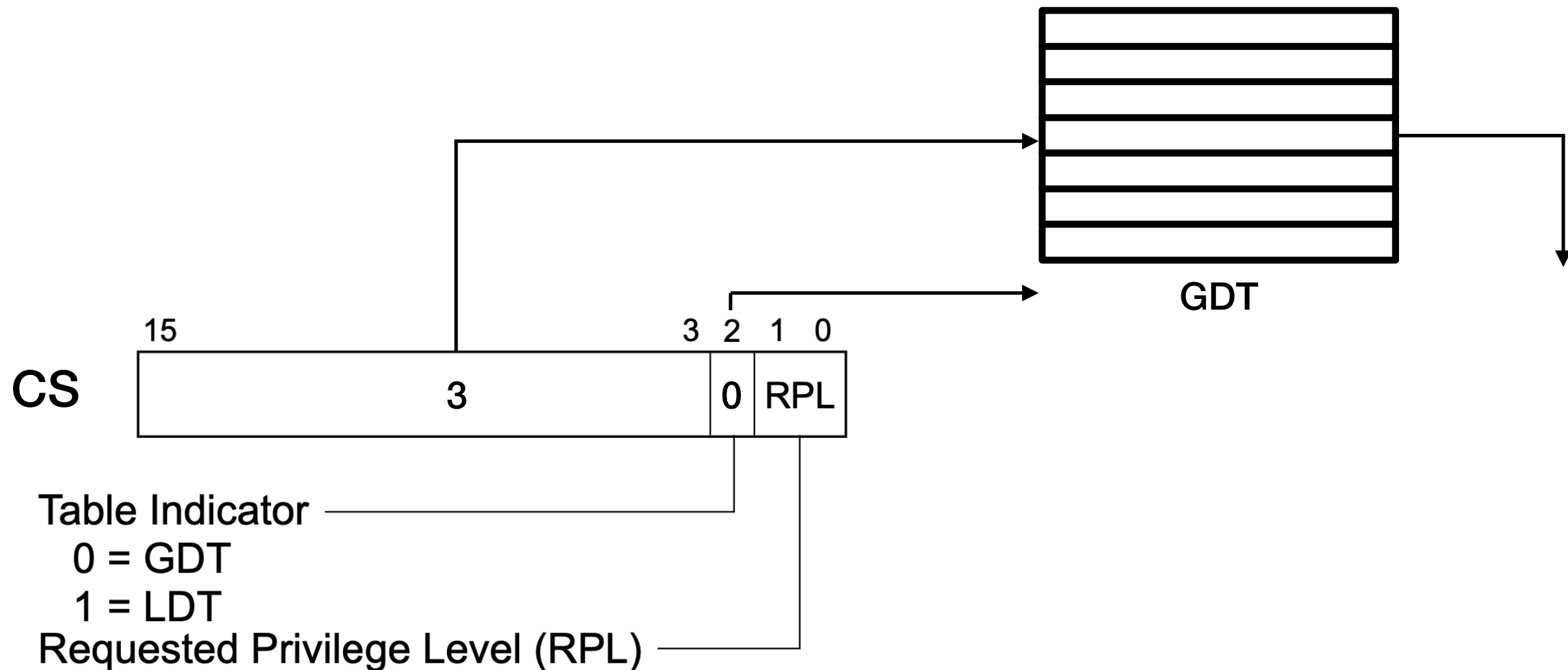


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

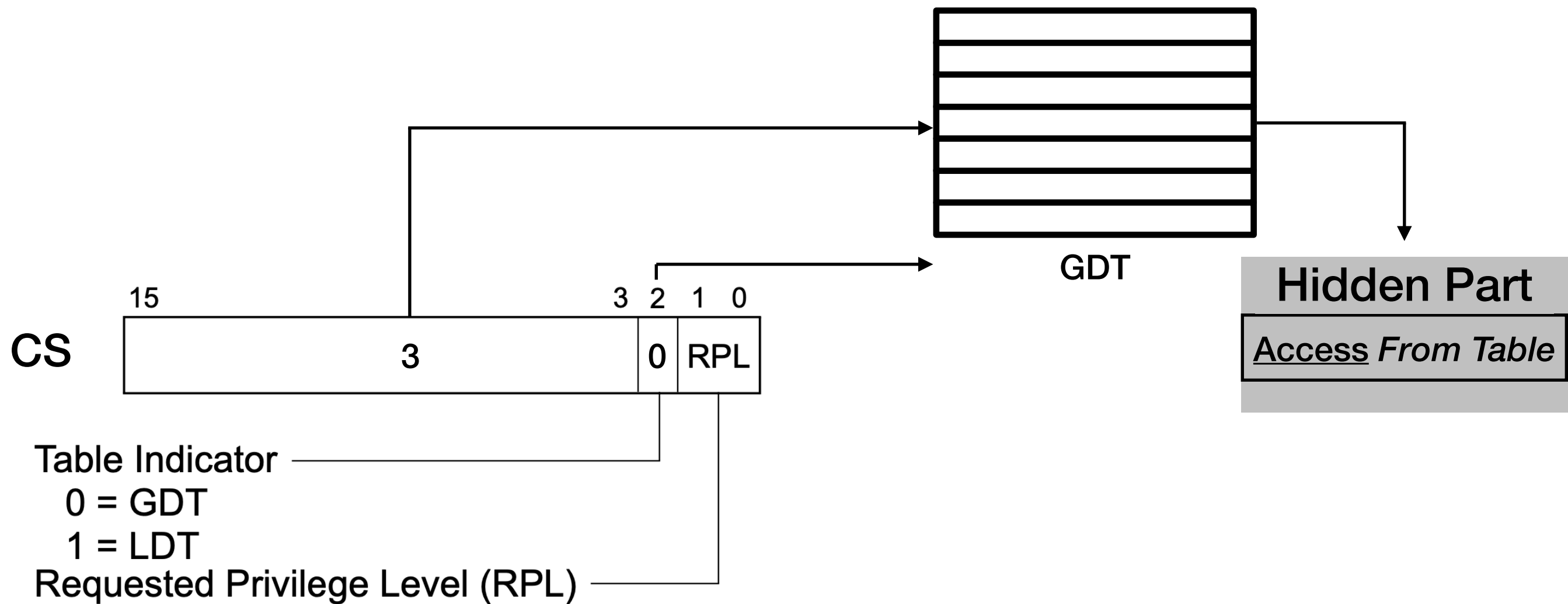


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

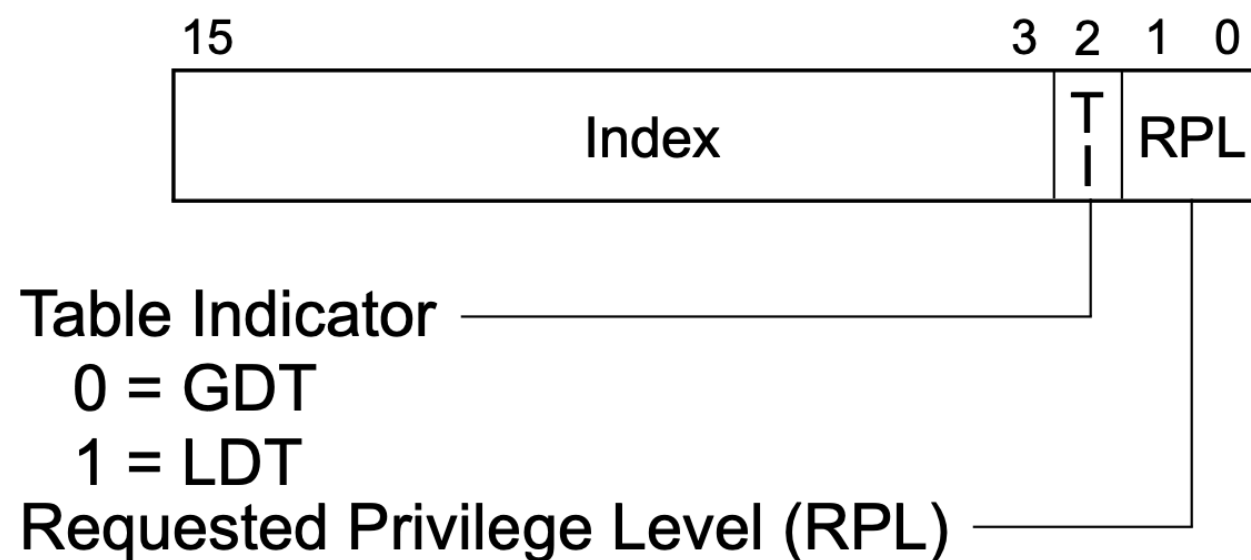


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

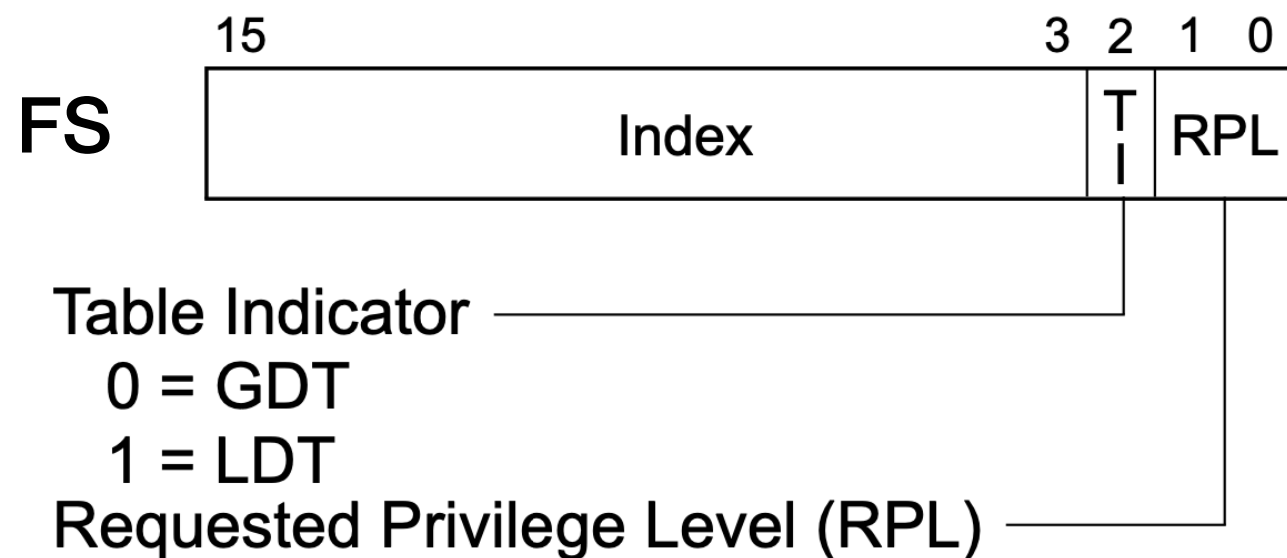


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

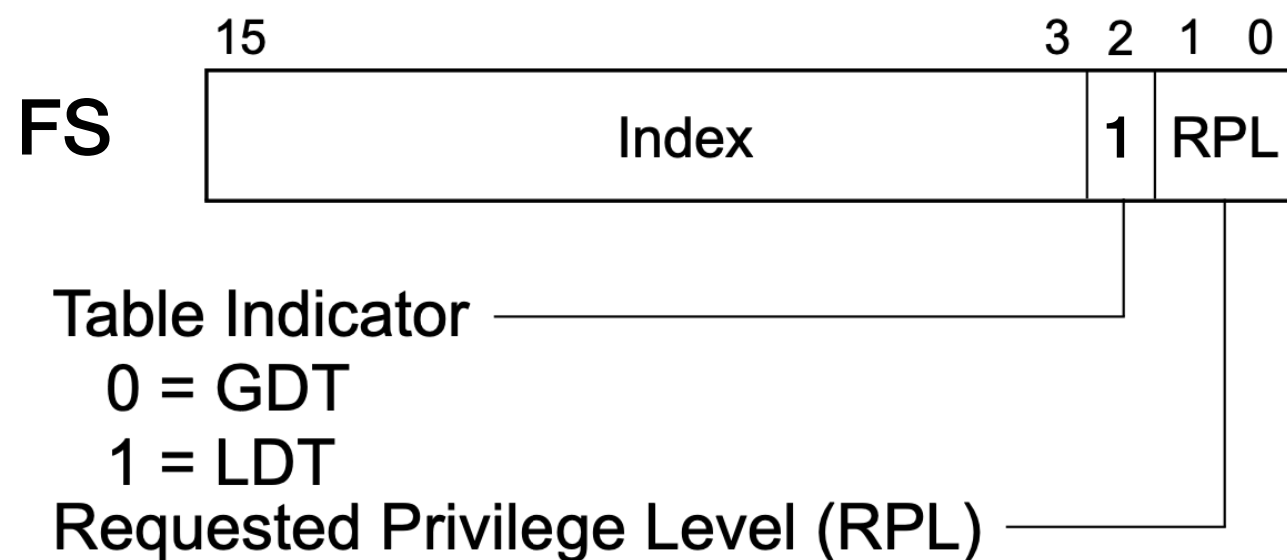


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

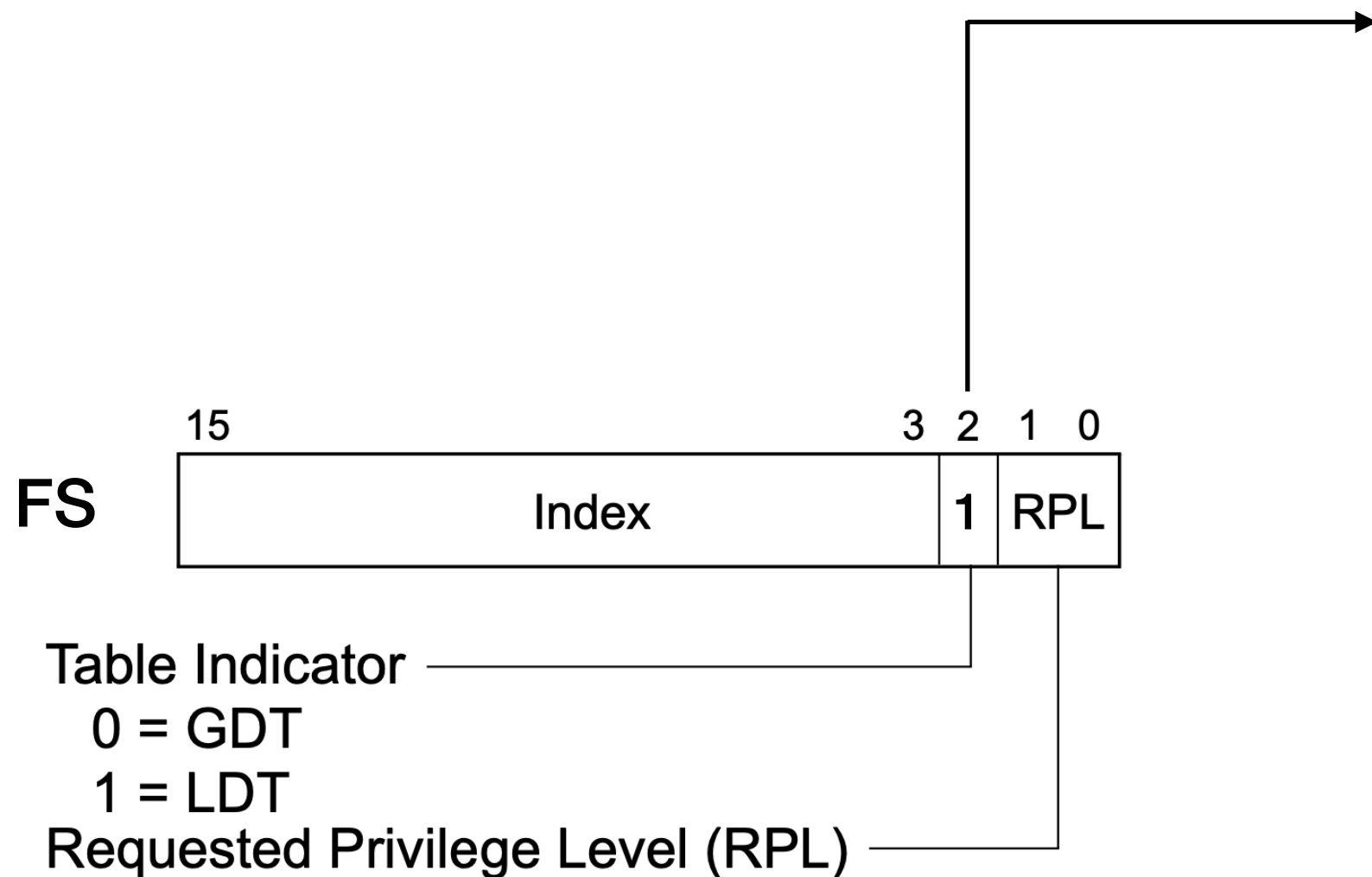


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

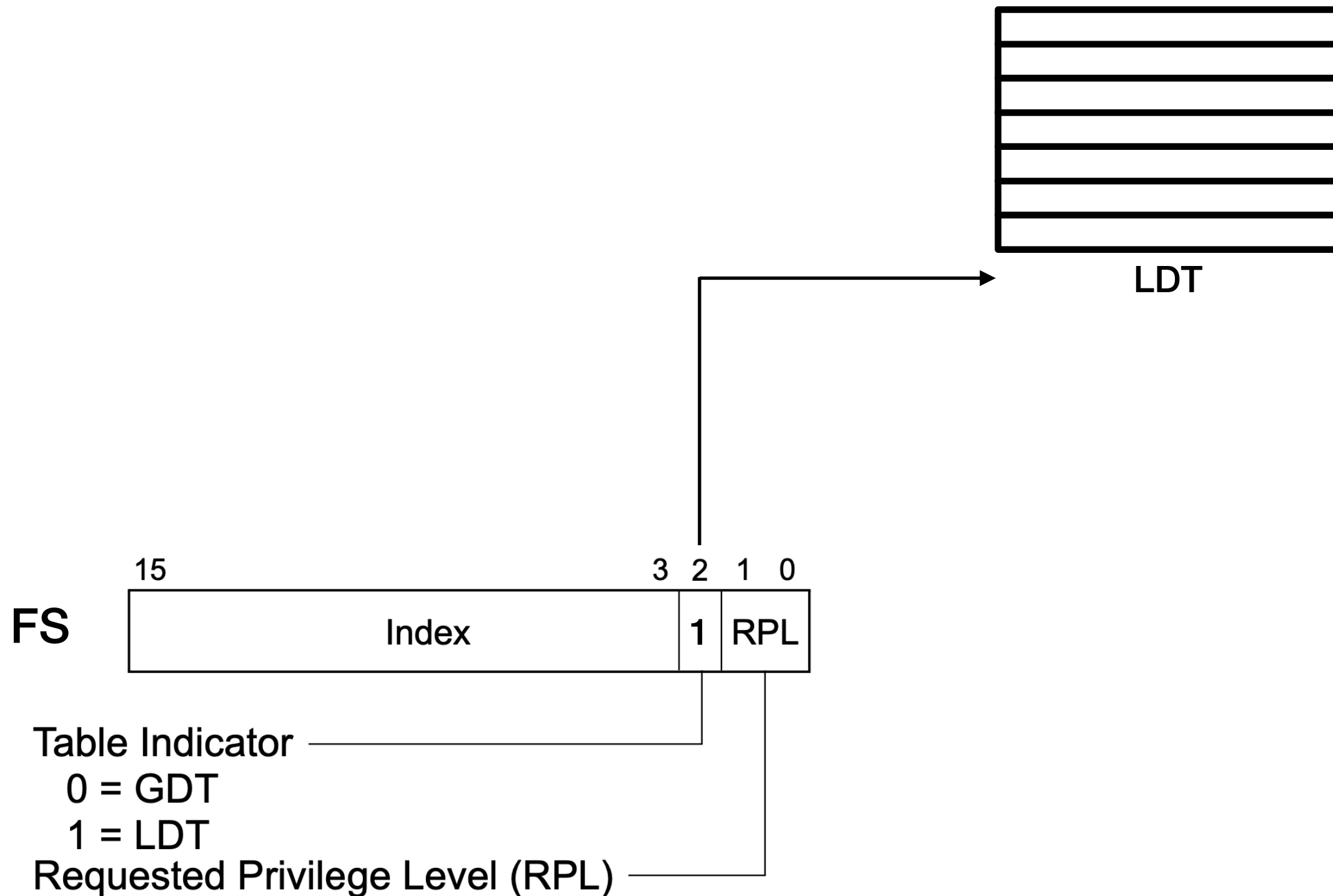


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

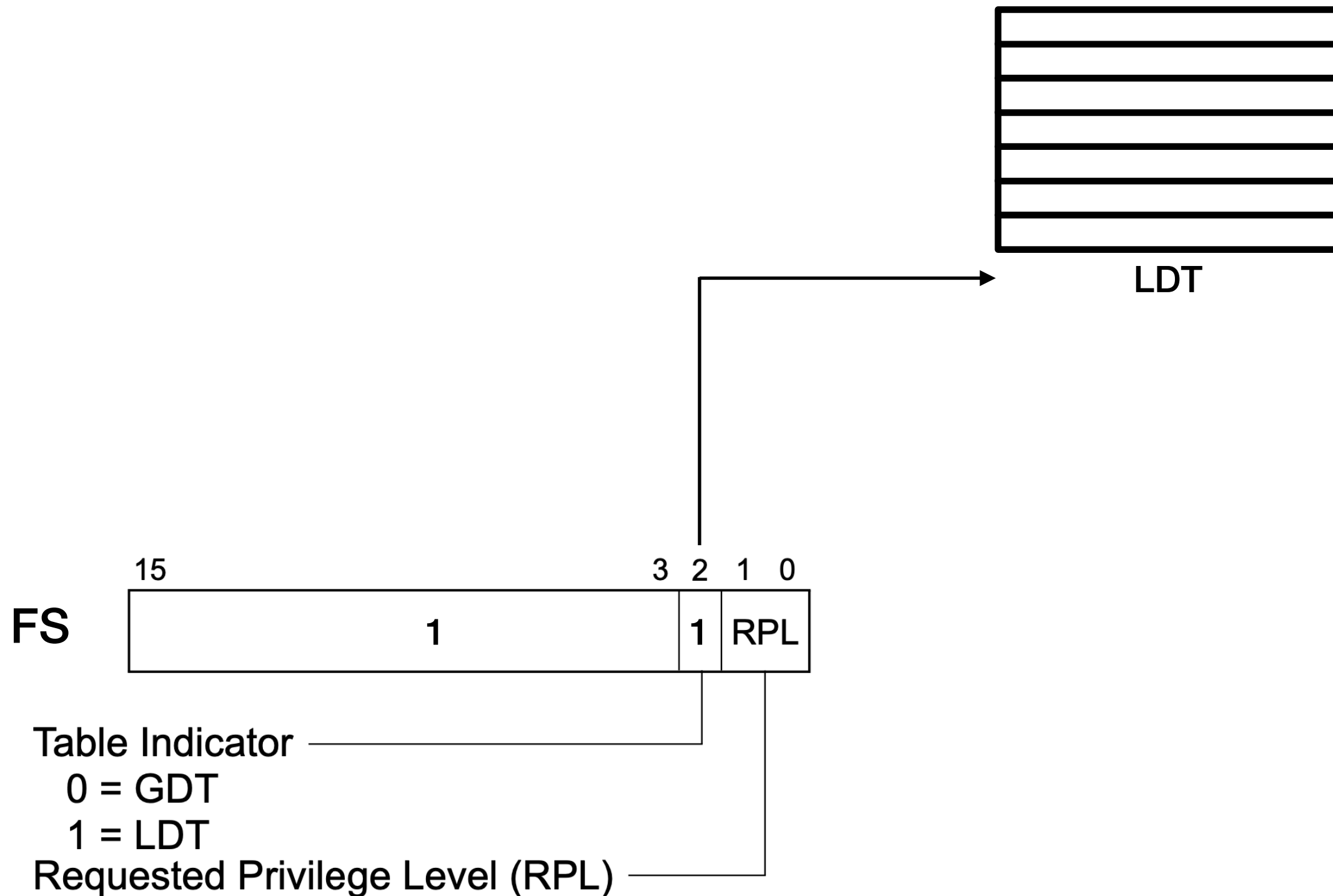


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

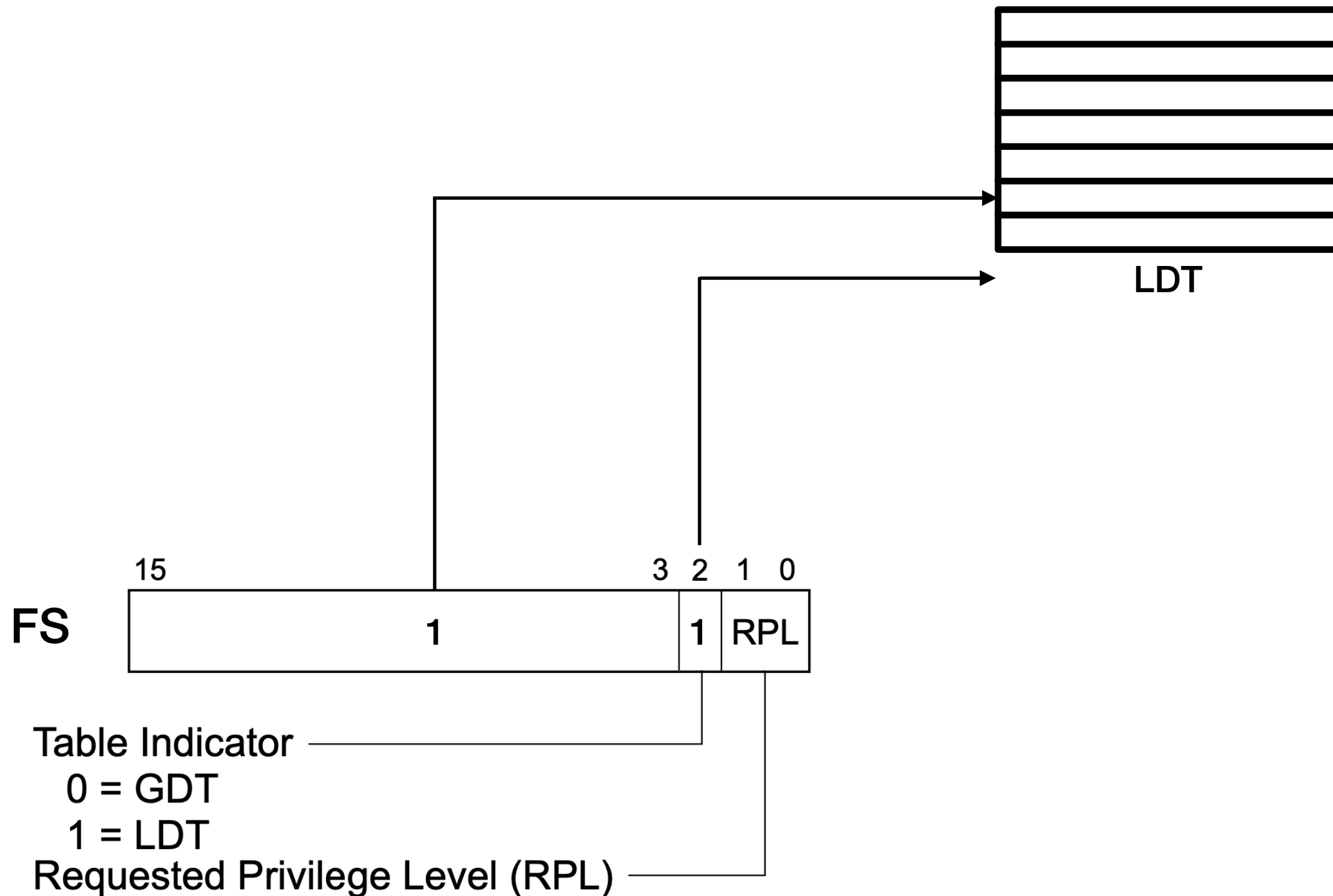


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

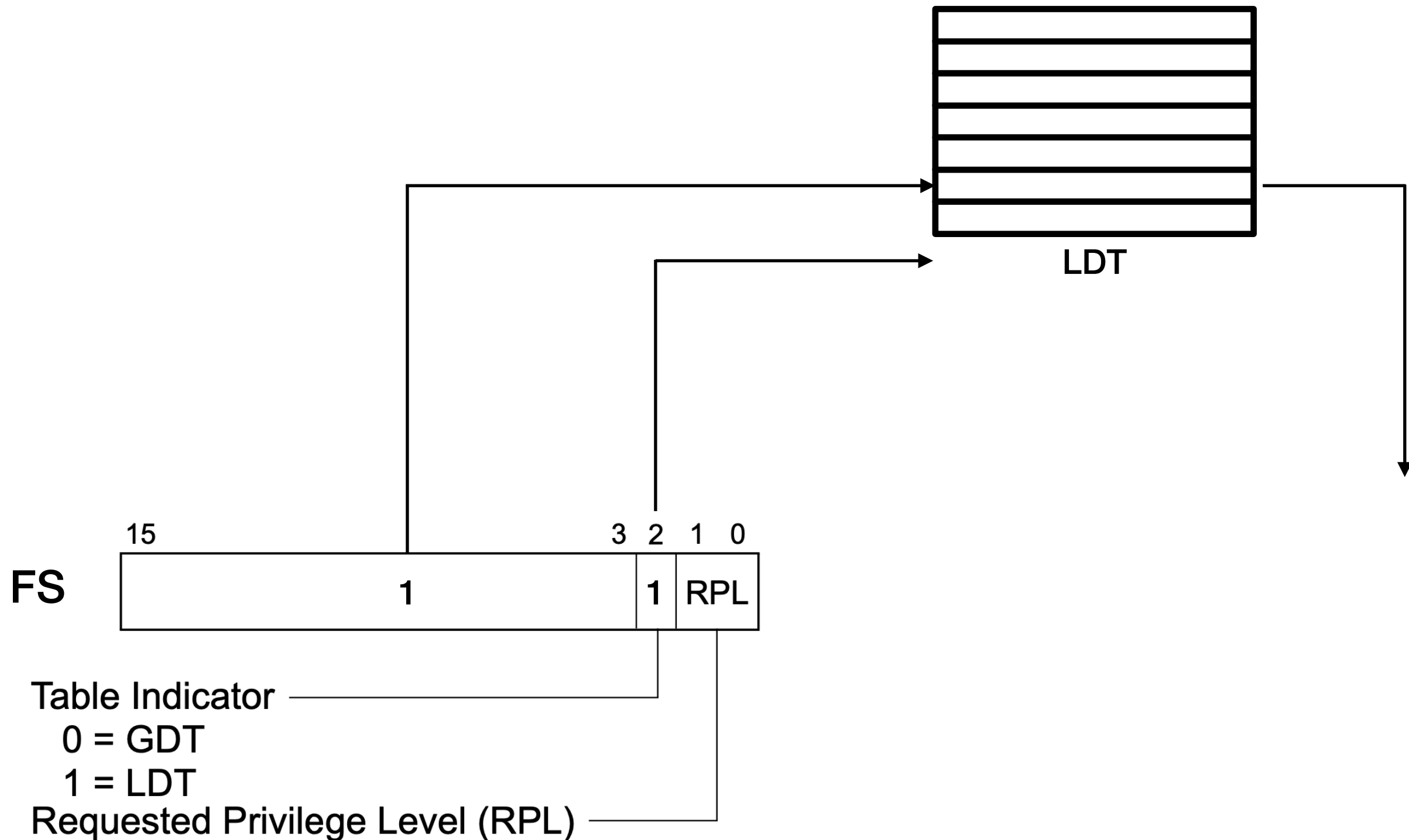


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

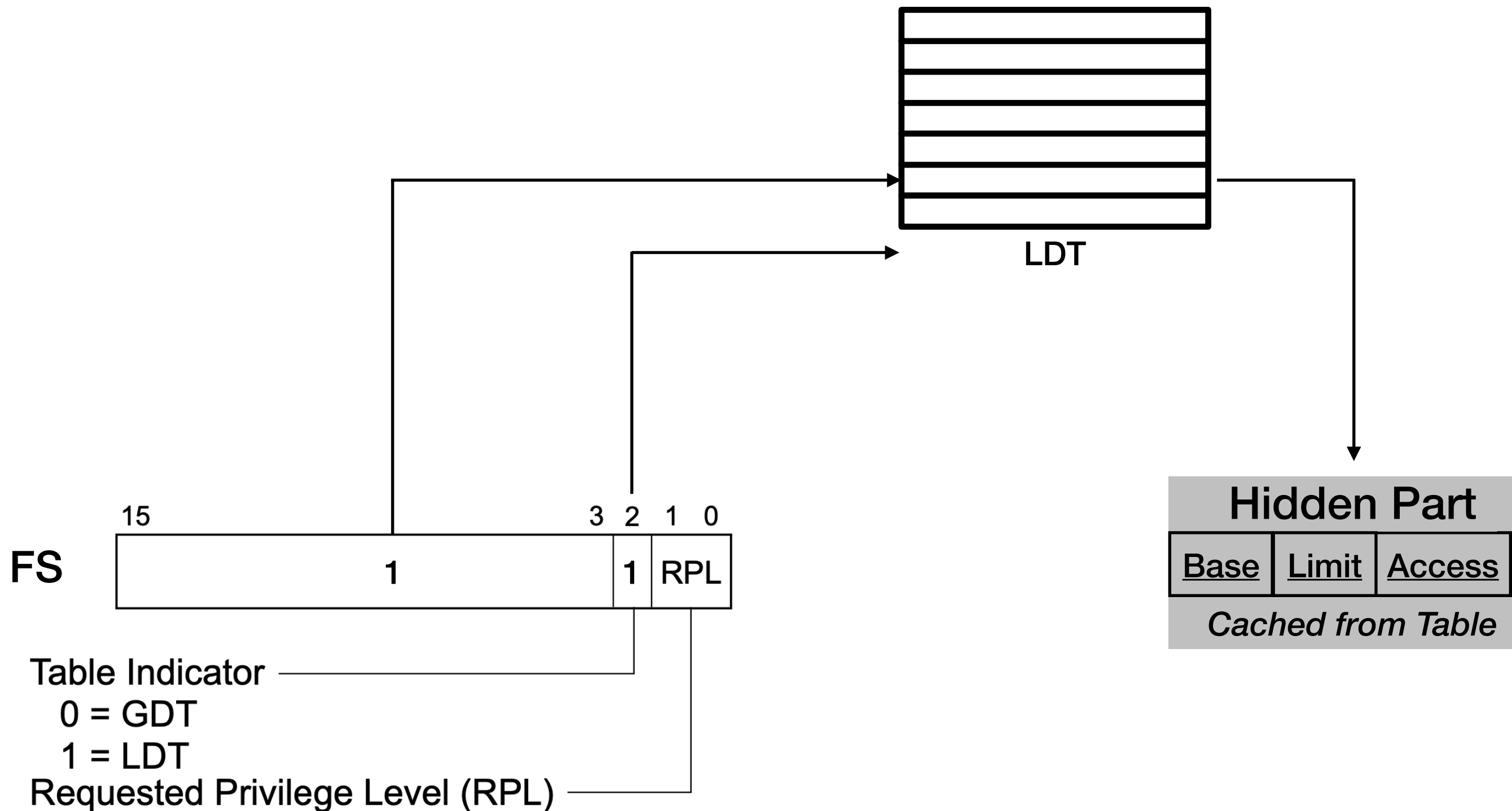


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...

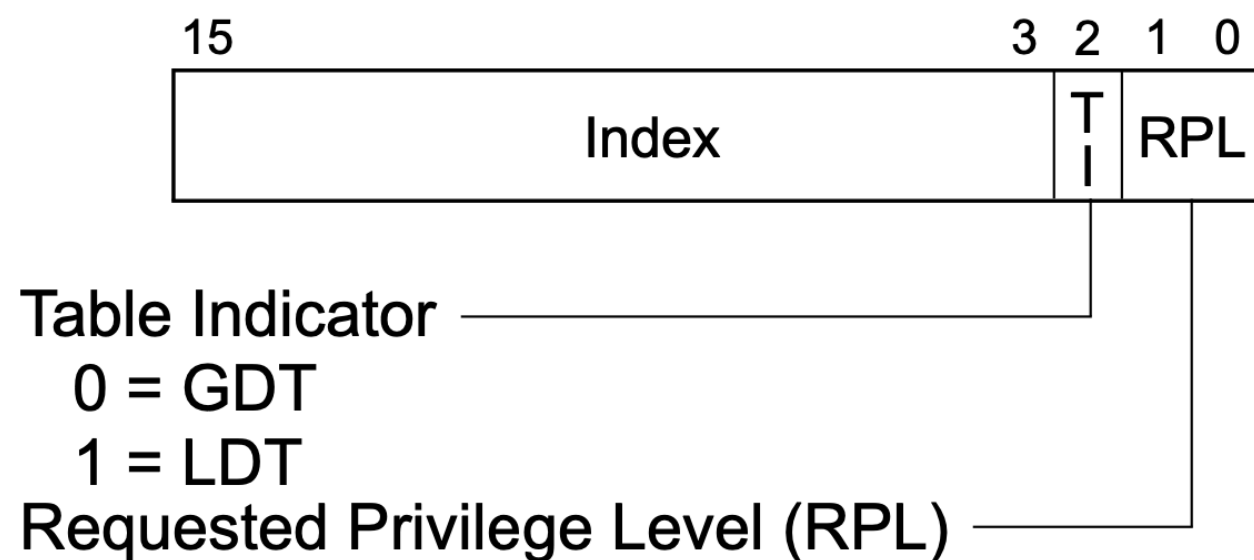
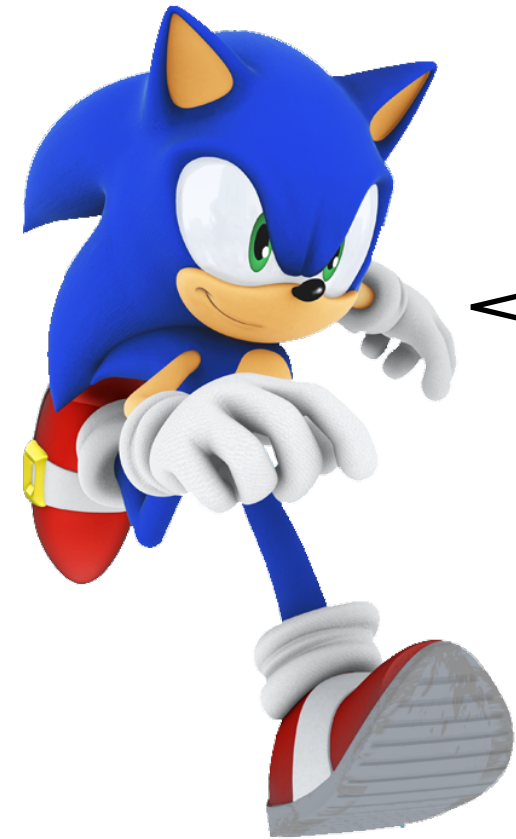


Figure 3-6. Segment Selector

What's Really Happening With x86-64 Segmentation...



Hypothesis!

If RPL has something to do with privilege rings, it should be different when reading segment selectors in kernel vs. userspace!

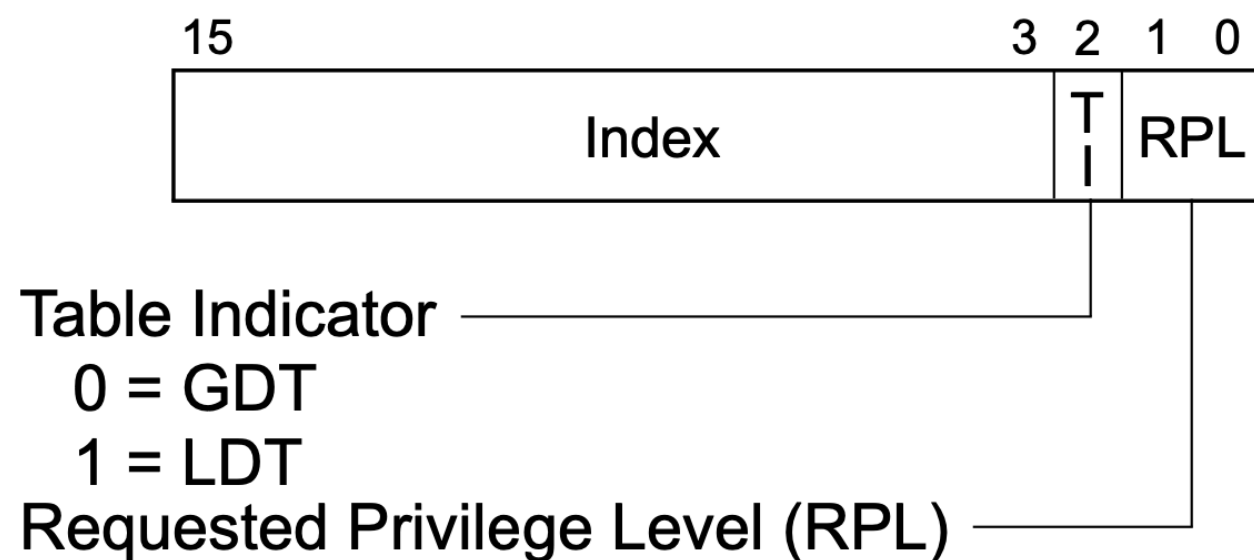


Figure 3-6. Segment Selector

Lab: U_SegRegs

- Userspace version of code which reads the segment
- Moves from segment registers to memory and then prints it out

Lab: K_SegRegs

- Kernel version of the same code as U_SegRegs

Results for Win10 20H2 x86-64

(subject to change on other versions, service pack levels, etc)

U_SegRegs

Segment Register	RPL	Table	Index
CS = 0x33	3	GDT	6
SS = 0x2b	3	GDT	5
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

K_SegRegs

Segment Register	RPL	Table	Index
CS = 0x10	0	GDT	2
SS = 0x18	0	GDT	3
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5



Results for Win10 20H2 x86-64

(subject to change on other versions, service pack levels, etc)

U_SegRegs

Segment Register	RPL	Table	Index
CS = 0x33	3	GDT	6
SS = 0x2b	3	GDT	5
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

!=

K_SegRegs

Segment Register	RPL	Table	Index
CS = 0x10	0	GDT	2
SS = 0x18	0	GDT	3
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5



Results for Win10 20H2 x86-64

(subject to change on other versions, service pack levels, etc)

U_SegRegs

Segment Register	RPL	Table	Index
CS = 0x33	3	GDT	6
SS = 0x2b	3	GDT	5
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

!=

==

K_SegRegs

Segment Register	RPL	Table	Index
CS = 0x10	0	GDT	2
SS = 0x18	0	GDT	3
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5



Inferences

- Windows maintains different CS, SS segment selectors for userspace processes vs kernel ones
- Windows x86-64 doesn't change DS, ES, FS, or GS *segment selectors* when moving between userspace and kernel (they were the exact same values)
- **The RPL field of the CS and SS segment selectors seems to correlate with the ring for kernel or userspace**





Logical to Linear One More Time

One of the segment registers
(SS/CS/DS/ES/FS/GS)

Address used in some
assembly instruction

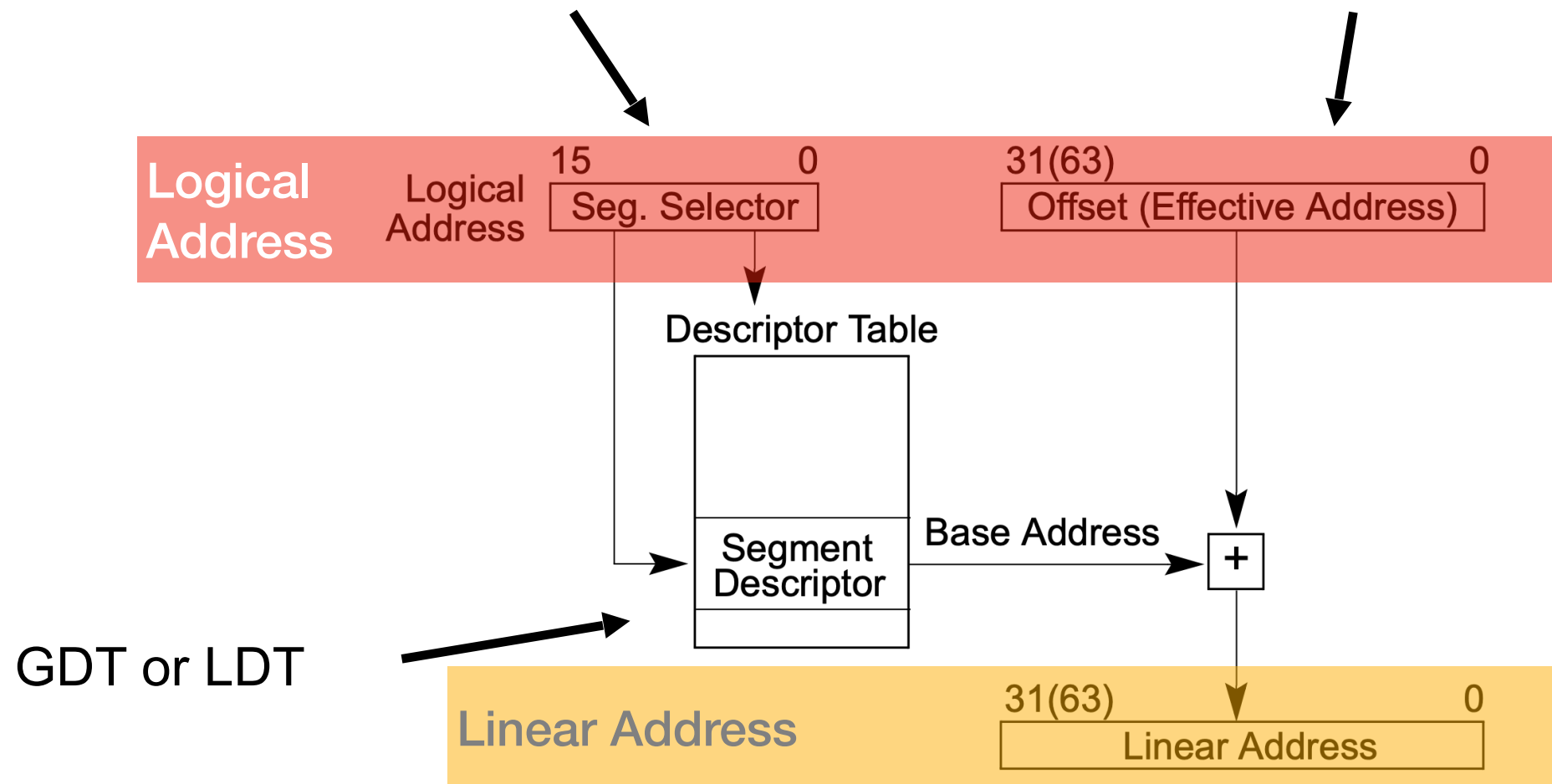
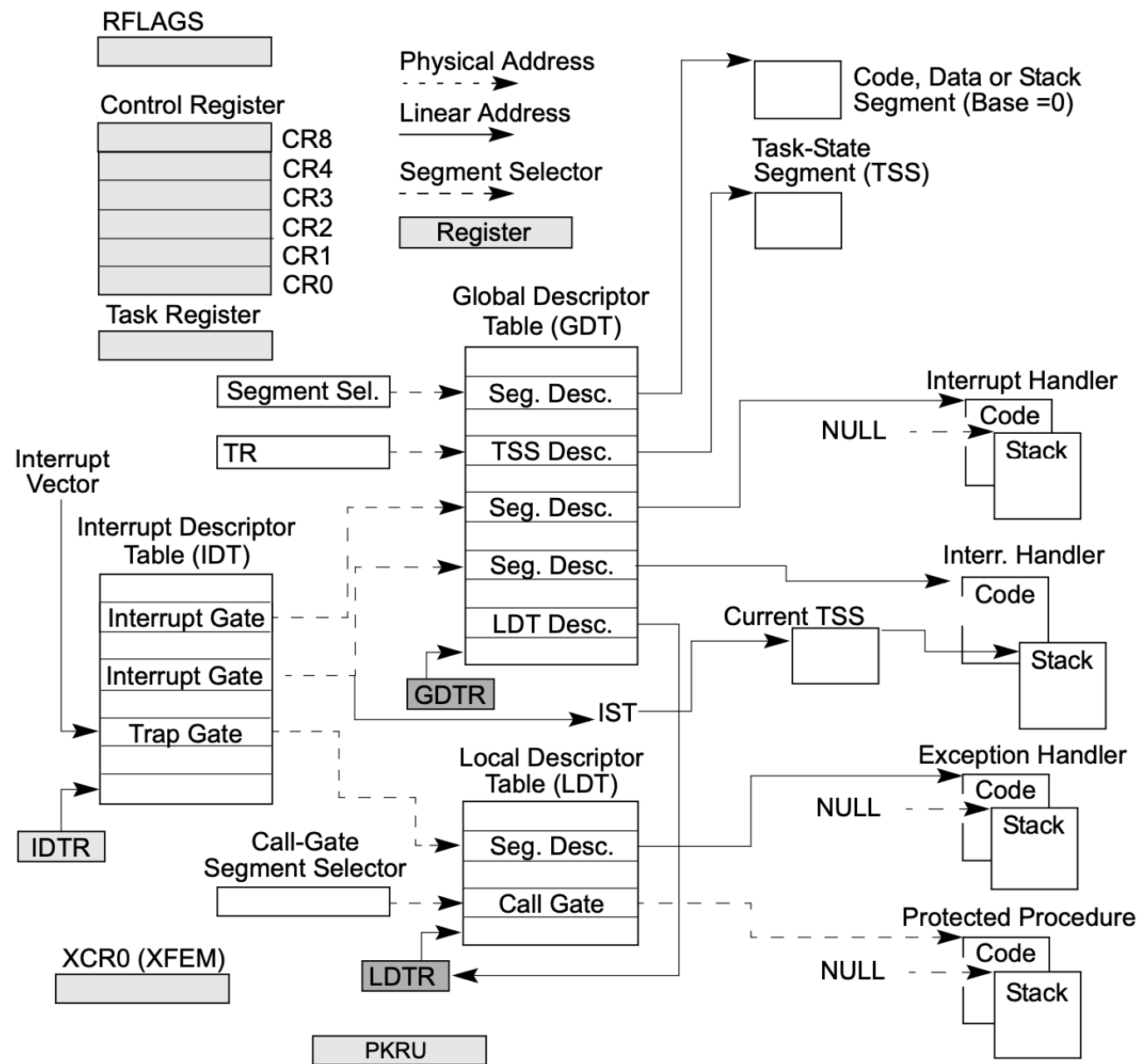
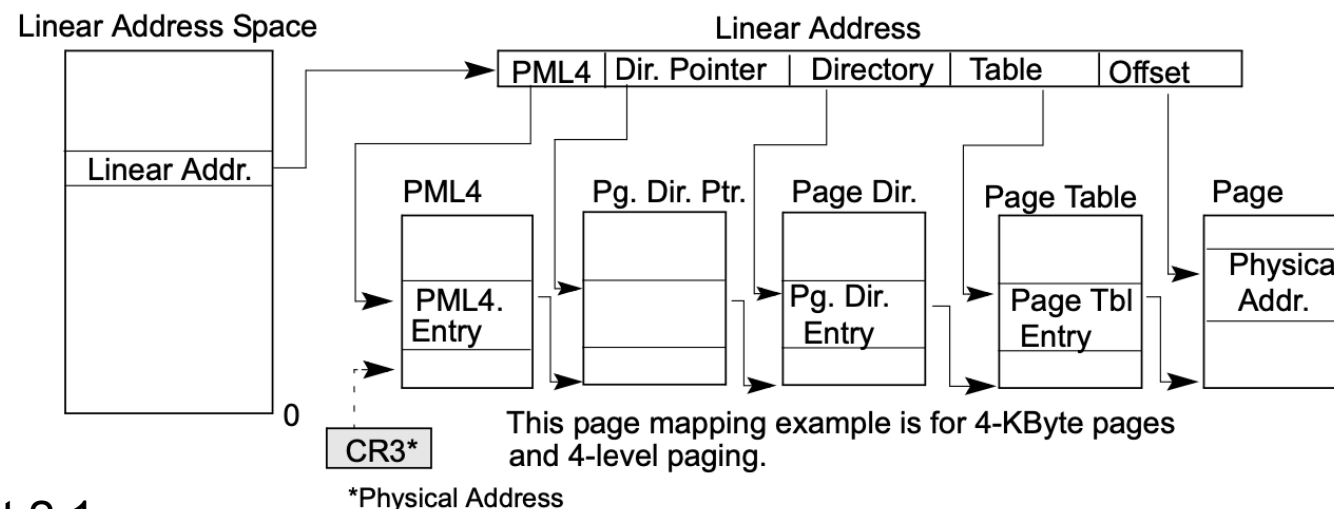
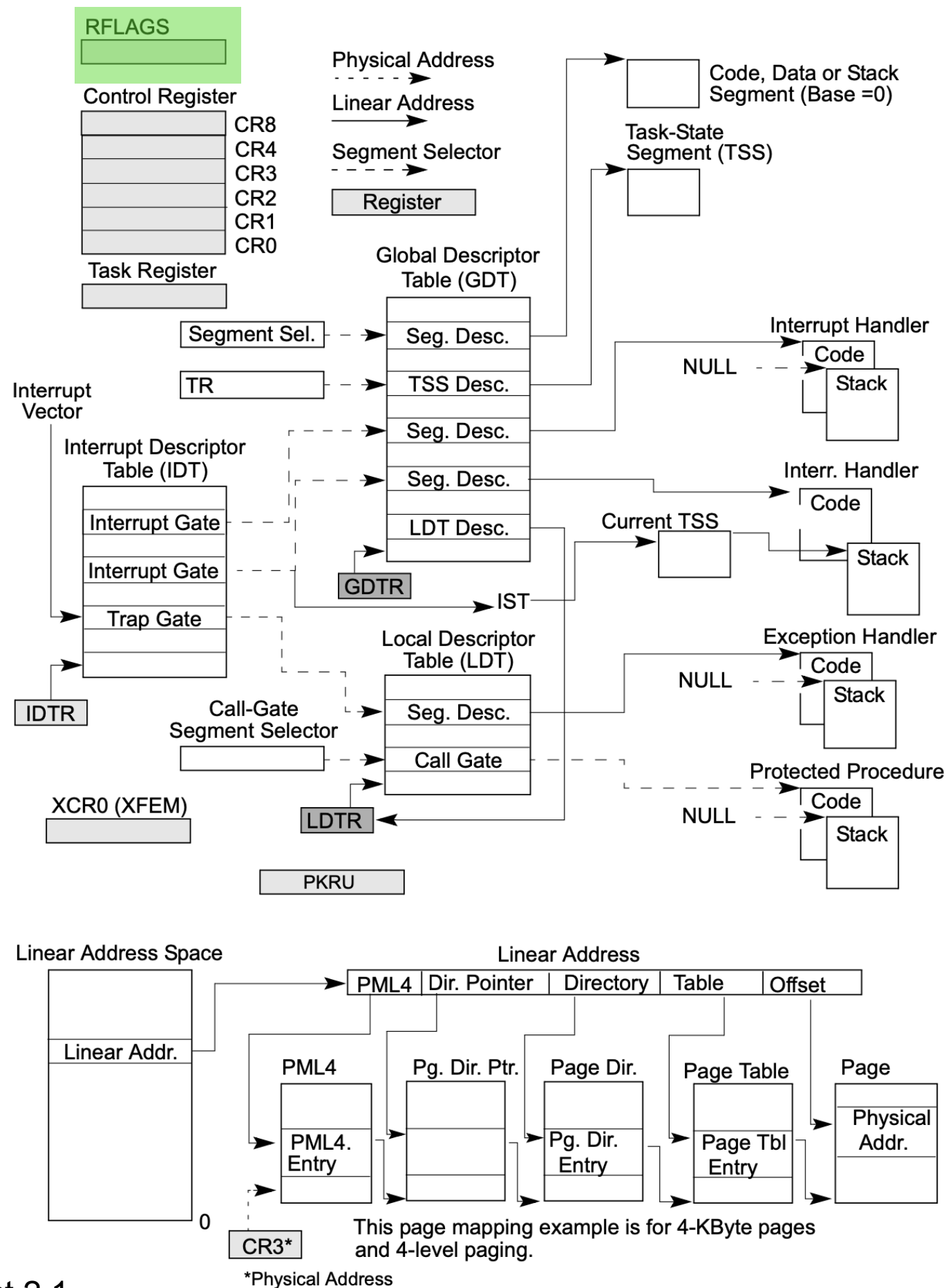


Figure 3-5. Logical Address to Linear Address Translation



← That's what you're going to learn! :D





That's what you're going to learn! :D

32-bit Segmentation **throwback**

- Optional!

Lab: U_SegRegs32

- Same as U_SegRegs, just 32 bit assembly for reading the segment registers
- Make sure you compile this as a Release Win32 build in VisualStudio, and then run it on a Windows 10 32-bit VM which has the VisualStudio 2019 redistributable package installed

Lab: K_SegRegs32

- Same as K_SegRegs, just 32 bit assembly for reading the segment registers
- Make sure you compile this as Win32 in VisualStudio, and then run it on a Windows 10 32-bit VM

Results for Win10 21H1 x86-32

(subject to change on other versions, service pack levels, etc)

U_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x1b	3	GDT	3
SS = 0x23	3	GDT	4
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x3B	3	GDT	7
GS = 0	Invalid	Invalid	Invalid

K_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x8	0	GDT	1
SS = 0x10	0	GDT	2
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x30	0	GDT	6
GS = 0	Invalid	Invalid	Invalid



Results for Win10 21H1 x86-32

(subject to change on other versions, service pack levels, etc)

U_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x1b	3	GDT	3
SS = 0x23	3	GDT	4
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x3B	3	GDT	7
GS = 0	Invalid	Invalid	Invalid

!=

K_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x8	0	GDT	1
SS = 0x10	0	GDT	2
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x30	0	GDT	6
GS = 0	Invalid	Invalid	Invalid



Results for Win10 21H1 x86-32

(subject to change on other versions, service pack levels, etc)

U_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x1b	3	GDT	3
SS = 0x23	3	GDT	4
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x3B	3	GDT	7
GS = 0	Invalid	Invalid	Invalid

K_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x8	0	GDT	1
SS = 0x10	0	GDT	2
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x30	0	GDT	6
GS = 0	Invalid	Invalid	Invalid

!=

==



Results for Win10 21H1 x86-32

(subject to change on other versions, service pack levels, etc)

U_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x1b	3	GDT	3
SS = 0x23	3	GDT	4
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x3B	3	GDT	7
GS = 0	Invalid	Invalid	Invalid

K_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x8	0	GDT	1
SS = 0x10	0	GDT	2
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x30	0	GDT	6
GS = 0	Invalid	Invalid	Invalid

!=

==



Results for Win10 21H1 x86-32

(subject to change on other versions, service pack levels, etc)

U_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x1b	3	GDT	3
SS = 0x23	3	GDT	4
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x3B	3	GDT	7
GS = 0	Invalid	Invalid	Invalid

K_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x8	0	GDT	1
SS = 0x10	0	GDT	2
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x30	0	GDT	6
GS = 0	Invalid	Invalid	Invalid

!=

==

!=



Results for Win10 21H1 x86-32

(subject to change on other versions, service pack levels, etc)

U_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x1b	3	GDT	3
SS = 0x23	3	GDT	4
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x3B	3	GDT	7
GS = 0	Invalid	Invalid	Invalid

K_SegRegs32

Segment Register	RPL	Table	Index
CS = 0x8	0	GDT	1
SS = 0x10	0	GDT	2
DS = 0x23	3	GDT	4
ES = 0x23	3	GDT	4
FS = 0x30	0	GDT	6
GS = 0	Invalid	Invalid	Invalid

!=

==

!=

==



Inferences

- Windows 32-bit maintains different CS, SS, & FS segment selectors for userspace processes vs kernel ones
- Windows doesn't change DS or ES when moving between userspace and kernel (they were the exact same values)
- Windows 32-bit doesn't use GS
- **The RPL field of the CS and SS segment selectors seems to correlate with the ring for kernel or userspace**

Differences from Win x86-64

- FS segment selector changes between userspace and kernel space in 32-bit, whereas it stayed the same on 64-bit
- GS segment selector is completely invalid in 32-bit, whereas it's valid but *seemingly* unchanging between userspace and kernel space in 64-bit